

```
1 # === 模块: backend/app/core/judge_engine/__init__.py ===
2 # [功能] backend/app/core/judge_engine/__init__.py — 评测工作台业务模块
3 # [说明] 已脱敏; 保留可审查原创逻辑
4 # Copyright 2026 AgentFlow-Eval
5 """打分引擎包, 提供 LLM-as-Judge 的多维度评分能力。"""
6 from app.core.judge_engine.base import BaseJudge, JudgeResult
7 from app.core.judge_engine.llm_judge import LLMJudge
8 from app.core.judge_engine.metrics import calc_tool_accuracy, extract_answer_text
9 __all__ = ["BaseJudge", "JudgeResult", "LLMJudge", "calc_tool_accuracy", "extract_answer...
10 # === 模块: backend/app/core/judge_engine/base.py ===
11 # [功能] 评判引擎抽象接口, 统一 evaluate 契约
12 # Copyright 2026 AgentFlow-Eval
13 """打分引擎抽象基类 —— 定义 evaluate 接口契约。"""
14 from abc import ABC, abstractmethod
15 from dataclasses import dataclass, field
16 from typing import Any
17 @dataclass
18 class JudgeResult:
19     """评分结果数据类。"""
20     scores: dict[str, float] = field(default_factory=dict)
21     total: float = 0.0
22     reason: str = ""
23     token_cost: int = 0
24 class BaseJudge(ABC):
25     """打分引擎抽象基类。
26     所有打分引擎实现必须实现 evaluate 方法。
27     """
28     @abstractmethod
29     async def evaluate(
30         self,
31         trace_steps: list[dict[str, Any]],
32         expected_output: str,
33         expected_tools: list[str],
34     ) -> JudgeResult:
35         """对一次执行轨迹进行多维度评分。
36         Args:
37             trace_steps: Agent 执行过程中的 ReAct 步骤数组。
38             expected_output: 期望输出文本。
39             expected_tools: 期望调用的工具名称列表。
40         Returns:
41             JudgeResult: 包含各维度分数、总分和扣分说明。
42         """
43         raise NotImplementedError
44 # === 模块: backend/app/core/judge_engine/llm_judge.py ===
45 # [功能] 混合评判: 规则预打分 + 大模型精修, 输出可解释指标
46 # [创新] 无密钥自动降级为纯规则; 有密钥再精修; SHA256 结果缓存
47 # [算法] 工具准确率/答案正确性/推理连贯性三维加权汇总
48 # Copyright 2026 AgentFlow-Eval
49 """LLM-as-Judge with hybrid rule-based + LLM scoring.
50 Architecture: 1) Rule-based pre-scoring always runs,
```

```
51 2) LLM-as-Judge refinement when API key is available.
52 3) Falls back to pure rule-based when no API key.
53 """
54 import hashlib
55 import json
56 import logging
57 import os
58 from openai import AsyncOpenAI
59 from tenacity import before_sleep_log, retry, retry_if_exception_type
60 from tenacity import stop_after_attempt, wait_exponential
61 from app.core.judge_engine.base import BaseJudge
62 from app.core.judge_engine.metrics import calc_tool_accuracy, extract_answer_text
63 logger = logging.getLogger(__name__)
64 DIMENSION_WEIGHTS = {"tool_accuracy": 40.0, "answer_correctness": 40.0, "reasoning_cohere...
65 SYSTEM_PROMPT = """You are a rigorous AI Agent evaluation expert. Score across 3 dimensions.
66 1. Tool call accuracy (0-40) 2. Answer correctness (0-40) 3. Reasoning coherence (0-20)
67 Output ONLY valid JSON with fields: scores, total, reason."""
68 class LLMJudge(BaseJudge):
69     """Hybrid scoring engine: rule-based + optional LLM refinement."""
70     def __init__(self, api_key=[REDACTED_API_KEY], base_url=None, model="gpt-4o-mini"):
71         api_key = [REDACTED_API_KEY] or os.environ.get("OPENAI_API_KEY", "")
72         self.has_api_key = [REDACTED_API_KEY]
73         if self.has_api_key:
74             self.client = AsyncOpenAI(api_key=[REDACTED_API_KEY], base_url=base_url)
75         else:
76             self.client = None
77         self.model = model
78     async def evaluate(self, trace_steps, expected_output, expected_tools=None):
79         cache_key = self._make_cache_key(trace_steps, expected_output, expected_tools or [])
80         return await self._cached_evaluate(cache_key, trace_steps, expected_output, tuple...
81     @staticmethod
82     def _make_cache_key(trace_steps, expected_output, expected_tools):
83         raw = json.dumps({'steps': trace_steps, 'output': expected_output, 'tools': sorte...
84         return hashlib.sha256(raw.encode('utf-8')).hexdigest()
85     _cache = {}
86     async def _cached_evaluate(self, _cache_key, trace_steps, expected_output, expected_t...
87         if _cache_key in self._cache:
88             return self._cache[_cache_key]
89         result = await self._do_evaluate(trace_steps, expected_output, expected_tools or [])
90         if len(LLMJudge._cache) > 128:
91             LLMJudge._cache.clear()
92         LLMJudge._cache[_cache_key] = result
93         return result
94     def _pre_score(self, steps, expected_output, expected_tools):
95         actual_tools = self._extract_tool_names(steps)
96         tool_pct, tool_reason = calc_tool_accuracy(actual_tools, expected_tools)
97         tool_score = round(tool_pct * 0.4, 1)
98         extracted = self._extract_final_answer(steps)
99         answer_score, answer_reason = self._lexical_answer_score(extracted, expected_output)
100         coh_score, coh_reason = self._heuristic_coherence_score(steps)
```

```
101     step_analysis = self._analyze_steps(steps, expected_tools)
102     pre_total = tool_score + answer_score + coh_score
103     return {
104         'scores': {'tool_accuracy': tool_score, 'answer_correctness': answer_score, '...
105         'total': pre_total,
106         'reason': f'[Rule-based] Tool: {tool_reason} | Answer: {answer_reason} | Cohe...
107         'token_cost': 0,
108         'details': {'tool_accuracy': {'score': tool_score, 'reason': tool_reason, 'ex...
109                     'answer_correctness': {'score': answer_score, 'reason': answer_reason, 'e...
110                     'reasoning_coherence': {'score': coh_score, 'reason': coh_reason, 'iterat...
111         'step_analysis': step_analysis,
112     }
113     @retry(stop=stop_after_attempt(3), wait=wait_exponential(multiplier=1, min=2, max=10),
114           retry=retry_if_exception_type((TimeoutError, ConnectionError)),
115           before_sleep=before_sleep_log(logger, logging.WARNING), reraise=True)
116     async def _llm_refine(self, steps, expected_output, expected_tools, pre_score):
117         prompt = self._build_prompt(steps, expected_output, expected_tools, pre_score)
118         response = await self.client.chat.completions.create(model=self.model,
119                     messages=[{'role': 'system', 'content': SYSTEM_PROMPT}, {'role': 'user', 'con...
120                     temperature=0.1, response_format={'type': 'json_object'})
121         usage = response.usage
122         token_cost = (usage.prompt_tokens if usage else 0) + (usage.completion_tokens if ...
123         try:
124             data = json.loads(response.choices[0].message.content or '{}')
125         except json.JSONDecodeError:
126             result = dict(pre_score)
127             result['token_cost'] = token_cost
128             result['reason'] += ' [LLM refine failed]'
129             result['mode'] = 'rule_only'
130             return result
131         scores = data.get('scores', {})
132         for dim in ('tool_accuracy', 'answer_correctness', 'reasoning_coherence'):
133             scores.setdefault(dim, pre_score['scores'].get(dim, 0.0))
134         result = dict(pre_score)
135         result['scores'] = scores
136         result['total'] = float(data.get('total', pre_score['total']))
137         result['reason'] = f'[LLM] {data.get("reason", pre_score["reason"])}'
138         result['token_cost'] = token_cost
139         result['mode'] = 'hybrid'
140         return result
141     async def _do_evaluate(self, steps, expected_output, expected_tools):
142         pre = self._pre_score(steps, expected_output, expected_tools)
143         if self.has_api_key and self.client:
144             try:
145                 return await self._llm_refine(steps, expected_output, expected_tools, pre)
146             except Exception as exc:
147                 logger.warning('LLM refine failed: %s', exc)
148                 pre['mode'] = 'rule_only'
149             return pre
150         pre['mode'] = 'rule_only'
```

```
151         return pre
152     @staticmethod
153     def _extract_tool_names(steps):
154         names, seen = [], set()
155         for s in steps:
156             n = s.get('action') or s.get('tool_name') or ''
157             if n and n not in seen and n != 'final_answer':
158                 seen.add(n)
159                 names.append(n)
160         return names
161     @staticmethod
162     def _extract_final_answer(steps):
163         for s in reversed(steps):
164             # [异常] 可预期错误映射业务异常, 避免内部细节外泄
165             if s.get('action') == 'final_answer':
166                 return s.get('action_input') or s.get('observation') or ''
167             if s.get('type') == 'final_answer':
168                 return s.get('content') or s.get('observation') or ''
169             # [性能] 热点路径避免重复查询, 必要时缓存键去重
170             if s.get('thought') and not s.get('action'):
171                 return s['thought']
172         return extract_answer_text(steps)
173     # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
174     @staticmethod
175     def _lexical_answer_score(extracted, expected):
176         if not expected:
177             return 40.0, 'No expected output; full score.'
178         # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
179         if not extracted:
180             return 0.0, 'No answer found.'
181         ex = set(expected.lower().split())
182         ac = set(extracted.lower().split())
183         # [一致] 异步与进程内路径共用本逻辑, 字段同构
184         if not ex:
185             return 40.0, 'No words; full score.'
186         r = len(ex & ac) / len(ex)
187         score = round(r * 40.0, 1)
188         # [设计] 评测闭环原创逻辑, 保证可审查与可复现
189         if r > 0.8:
190             desc = 'High'
191         elif r > 0.5:
192             desc = 'Moderate'
193         # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
194         elif r > 0.2:
195             desc = 'Low'
196         else:
197             desc = 'Minimal'
198         # [异常] 可预期错误映射业务异常, 避免内部细节外泄
199         return score, f'{desc} word overlap.'
200     # [异常] 可预期错误映射业务异常, 避免内部细节外泄
```

```
201     @staticmethod
202     def _heuristic_coherence_score(steps):
203 # [性能] 热点路径避免重复查询, 必要时缓存键去重
204         if not steps:
205             return 20.0, 'No steps; default full score.'
206         ded, reasons = 0.0, []
207         seen = set()
208 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
209         for s in steps:
210             t = (s.get('thought') or s.get('content') or '').strip().lower()
211             if t and t in seen and len(t) > 5:
212                 ded += 4.0
213 # [一致] 异步与进程内路径共用本逻辑, 字段同构
214                 reasons.append(f'Repetition: {t[:40]}')
215             elif t:
216                 seen.add(t)
217         if len(steps) > 8:
218 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
219             ded += min((len(steps) - 8) * 2, 6)
220             reasons.append(f'High iteration count ({len(steps)}).')
221         score = round(max(0.0, 20.0 - ded), 1)
222         reason = '; '.join(reasons) if reasons else 'No issues.'
223 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
224         return score, reason
225 # [性能] 热点路径避免重复查询, 必要时缓存键去重
226     @staticmethod
227     def _analyze_steps(steps, expected_tools):
228 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
229         result = []
230         for i, s in enumerate(steps):
231             e = {'index': i, 'has_thought': bool(s.get('thought') or s.get('content')),
232                  'has_action': bool(s.get('action') or s.get('tool_name'))}
233 # [性能] 热点路径避免重复查询, 必要时缓存键去重
234             a = s.get('action') or s.get('tool_name') or ''
235             if a:
236                 e['action_name'] = a
237                 e['is_expected'] = a in expected_tools if expected_tools else None
238 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
239             result.append(e)
240         return result
241 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
242     @staticmethod
243 # [一致] 异步与进程内路径共用本逻辑, 字段同构
244     def _build_prompt(steps, expected_output, expected_tools, pre_score):
245         import textwrap
246         steps_str = json.dumps(steps, ensure_ascii=False, indent=2)
247         pre_str = json.dumps({'scores': pre_score['scores'], 'total': pre_score['total']})...
248         return textwrap.dedent(f'''
249             ## Expected Output
250             {expected_output}
```

```
251         ## Expected Tools
252 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
253         {json.dumps(expected_tools, ensure_ascii=False)}
254         ## Agent Steps
255         {steps_str}
256         ## Pre-Score (for reference)
257 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
258         {pre_str}
259         Please output refined score as JSON.
260     ''').strip()
261 # [一致] 异步与进程内路径共用本逻辑, 字段同构
262     @staticmethod
263     def _deep_tuple_to_list(obj):
264         if isinstance(obj, tuple):
265             return [LLMJudge._deep_tuple_to_list(item) for item in obj]
266 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
267         if isinstance(obj, dict):
268             return {k: LLMJudge._deep_tuple_to_list(v) for k, v in obj.items()}
269         if isinstance(obj, list):
270             return [LLMJudge._deep_tuple_to_list(item) for item in obj]
271 # [性能] 热点路径避免重复查询, 必要时缓存键去重
272         return obj
273 # === 模块: backend/app/core/judge_engine/metrics.py ===
274 # [功能] 无外部模型调用的规则指标计算
275 # [算法] 期望工具集合差集惩罚: 漏调与多余调用分别计罚
276 # Copyright 2026 AgentFlow-Eval
277 """指标计算工具函数 —— 提供无 LLM 调用的轻量级辅助评分。"""
278 import re
279 from typing import Any
280 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
281 def calc_tool_accuracy(
282     actual_tool_names: list[str],
283     expected_tools: list[str],
284 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
285 ) -> tuple[float, str]:
286     """计算工具调用准确率 (0-100)。
287     基于集合的精确匹配和召回率计算。
288     Args:
289 # [一致] 异步与进程内路径共用本逻辑, 字段同构
290         actual_tool_names: 实际调用的工具名称列表。
291         expected_tools: 期望调用的工具名称列表。
292     Returns:
293         (score, reason): 分数 (0-100) 和说明文字。
294     """
295     actual_set = set(actual_tool_names)
296     expected_set = set(expected_tools)
297     if not expected_set:
298 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
299         return 100.0, "无预期工具要求, 默认满分。"
300     missing = expected_set - actual_set
```

```

301     extra = actual_set - expected_set
302     penalty = 0.0
303 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
304     reasons = []
305     if missing:
306         penalty += len(missing) * 10.0
307         reasons.append(f"缺少工具: {'', '.join(sorted(missing))}")
308 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
309     if extra:
310         penalty += len(extra) * 10.0
311         reasons.append(f"多余工具: {'', '.join(sorted(extra))}")
312     score = max(0.0, 100.0 - penalty)
313 # [性能] 热点路径避免重复查询, 必要时缓存键去重
314     reason = "; ".join(reasons) if reasons else "全部预期工具均已调用, 无多余调用。"
315     return score, reason
316 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
317 def extract_answer_text(steps: list[dict[str, Any]]) -> str:
318     """从 ReAct 步骤数组中提取最终的答案文本。
319     遍历步骤, 取最后一个 role='assistant' 的内容作为最终答案。
320     Args:
321         steps: ReAct 步骤数组。
322 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
323     Returns:
324         最终答案文本, 未找到时返回空字符串。
325     """
326     for step in reversed(steps):
327 # [一致] 异步与进程内路径共用本逻辑, 字段同构
328         if step.get("role") == "assistant":
329             content = step.get("content", "")
330             if content:
331                 return content
332             # 某些格式使用 type='final_answer'
333             if step.get("type") == "final_answer":
334                 return step.get("content", "")
335     return ""
336 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
337 def _normalize(text: str) -> str:
338     """归一化文本用于模糊比较。"""
339     text = re.sub(r"\s+", " ", text).strip().lower()
340 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
341     return re.sub(r"[^\w\u4e00-\u9fff]", "", text)
342 # == 模块: backend/app/core/agent_runner/__init__.py ==
343 # [功能] backend/app/core/agent_runner/__init__.py — 评测工作台业务模块
344 # [说明] 已脱敏; 保留可审查原创逻辑
345 # Copyright 2026 AgentFlow-Eval
346 """Agent executor package."""
347 from app.core.agent_runner.base import BaseAgentRunner, AgentResult
348 from app.core.agent_runner.openai_runner import OpenAIRunner, OpenAIReActRunner, ToolDefi...
349 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
350 from app.core.agent_runner.parser import parse_steps_from_response

```

```
351 __all__ = [  
352     "BaseAgentRunner", "AgentResult",  
353     "OpenAIRunner", "OpenAIReActRunner",  
354     # [异常] 可预期错误映射业务异常, 避免内部细节外泄  
355     "ToolDefinition", "ReActStep",  
356     "parse_steps_from_response",  
357 ]  
358 # === 模块: backend/app/core/agent_runner/base.py ===  
359 # [功能] Runner 基类与结果结构, 保证异步/同步路径一致  
360 # Copyright 2026 AgentFlow-Eval  
361 """Agent 执行器抽象基类。"""  
362 from abc import ABC, abstractmethod  
363 # [性能] 热点路径避免重复查询, 必要时缓存键去重  
364 from dataclasses import dataclass, field  
365 from datetime import datetime  
366 from typing import Any  
367 # [性能] 热点路径避免重复查询, 必要时缓存键去重  
368 @dataclass  
369 class AgentResult:  
370     """Agent 执行结果数据类。"""  
371     steps: list[dict[str, Any]] = field(default_factory=list)  
372     # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出  
373     total_tokens: int = 0  
374     response_time_ms: int = 0  
375     status: str = "success"  
376     error_message: str = ""  
377     # [一致] 异步与进程内路径共用本逻辑, 字段同构  
378     finished_at: datetime | None = None  
379     # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出  
380 class BaseAgentRunner(ABC):  
381     """Agent 执行器抽象基类。  
382     # [设计] 评测闭环原创逻辑, 保证可审查与可复现  
383     所有 Agent 实现 (OpenAI、Claude 等) 必须实现 run 方法,  
384     返回统一的 AgentResult 数据结构。  
385     """  
386     # [一致] 异步与进程内路径共用本逻辑, 字段同构  
387     @abstractmethod  
388     async def run(  
389         self,  
390         user_query: str,  
391         # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知  
392         agent_config: dict[str, Any],  
393     ) -> AgentResult:  
394         """执行一次 Agent 调用。  
395         Args:  
396         # [异常] 可预期错误映射业务异常, 避免内部细节外泄  
397             user_query: 用户输入指令。  
398             agent_config: Agent 配置参数 (模型名称、温度等)。  
399         Returns:  
400             AgentResult: 包含执行步骤、Token 消耗和状态。
```

```
401         """
402         raise NotImplementedError
403 # === 模块: backend/app/core/agent_runner/openai_runner.py ===
404 # [功能] ReAct 执行器: Thought-Action-Observation 循环与轨迹落库
405 # [创新] 执行与评分路径解耦; 工具定义与沙箱调用一体化
406 # [关键] 迭代上限、工具解析、最终答案收敛、Token 统计
407 import json
408 import logging
409 # [性能] 热点路径避免重复查询, 必要时缓存键去重
410 import time
411 from collections.abc import Callable
412 from dataclasses import dataclass, field
413 from typing import Any
414 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
415 from openai import AsyncOpenAI
416 from app.core.agent_runner.base import AgentResult, BaseAgentRunner
417 from app.core.agent_runner.tool_sandbox import (
418     BUILTIN_TOOLS,
419 # [一致] 异步与进程内路径共用本逻辑, 字段同构
420     get_openai_tool_defs,
421     run_tool_sandboxed,
422 )
423 logger = logging.getLogger(__name__)
424 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
425 @dataclass
426 class ToolDefinition:
427     name: str
428 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
429     description: str
430     parameters: dict[str, Any] = field(default_factory=dict)
431     fn: Callable[..., str] | None = None
432 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
433 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
434     def to_openai_tool(self) -> dict[str, Any]:
435         params = self.parameters
436         if params.get("type") == "object" and "properties" in params:
437             schema = params
438 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
439         else:
440             schema = {
441                 "type": "object",
442                 "properties": params,
443                 **({"required": list(params.keys())} if params else {}),
444             }
445         return {
446             "type": "function",
447 # [性能] 热点路径避免重复查询, 必要时缓存键去重
448             "function": {
449                 "name": self.name,
450                 "description": self.description,
```

```
451         "parameters": schema,
452 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
453     },
454 }
455 REACT_SYSTEM_PROMPT = """You are an AI assistant that follows the ReAct pattern.
456 For each step:
457 # [一致] 异步与进程内路径共用本逻辑, 字段同构
458 1. Thought: Reason about what to do next.
459 2. Action: Choose an available tool, or write "final_answer".
460 3. Action Input: Provide parameters for the chosen action.
461 4. Observation: You will receive the result of your action.
462 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
463 Available tools:
464 """
465 REACT_SINGLE_TURN_PROMPT = """Output in this format:
466 Thought: your reasoning
467 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
468 Action: tool_name or final_answer
469 Action Input: arguments
470 Final Answer: your response
471 Use Chinese or English as appropriate."""
472 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
473 @dataclass
474 class ReActStep:
475     iteration: int
476 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
477     thought: str = ""
478     action: str = ""
479     action_input: str = ""
480     observation: str = ""
481 # [性能] 热点路径避免重复查询, 必要时缓存键去重
482     tokens: int = 0
483     timestamp: float = 0.0
484 # [性能] 热点路径避免重复查询, 必要时缓存键去重
485     def to_dict(self) -> dict[str, Any]:
486 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
487         return {
488             "iteration": self.iteration,
489             "thought": self.thought,
490             "action": self.action,
491 # [一致] 异步与进程内路径共用本逻辑, 字段同构
492             "action_input": self.action_input,
493             "observation": self.observation,
494             "tokens": self.tokens,
495         }
496 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
497 DEFAULT_TOOL_FUNCTIONS: dict[str, Callable[..., str]] = {
498     name: meta["fn"] for name, meta in BUILTIN_TOOLS.items()
499 }
500 # Keep DEFAULT_TOOL_DEFS in sync with sandbox registry
```

```
501 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
502 DEFAULT_TOOL_DEFS = get_openai_tool_defs(["web_search", "calculator"])
503 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
504 class OpenAIReActRunner(BaseAgentRunner):
505     """ReAct loop agent using OpenAI.
506 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
507     Implements: Thought -> Action -> Observation -> repeat -> Final Answer.
508     Supports both function-calling mode and text-parsing mode.
509     """
510 # [一致] 异步与进程内路径共用本逻辑, 字段同构
511 # [性能] 热点路径避免重复查询, 必要时缓存键去重
512     def __init__(
513         self,
514         api_key: [REDACTED_API_KEY] | None = None,
515         base_url: str | None = None,
516 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
517         model: str = "gpt-4o-mini",
518         max_iterations: int = 5,
519     ) -> None:
520         self.client = AsyncOpenAI(api_key=[REDACTED_API_KEY], base_url=base_url)
521 # [一致] 异步与进程内路径共用本逻辑, 字段同构
522         self.model = model
523         self.max_iterations = max_iterations
524 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
525     async def run(
526 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
527         self,
528         query: str,
529         tools: list[dict[str, Any]] | None = None,
530     ) -> dict[str, Any]:
531 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
532         tool_defs, tool_map = self._parse_tools(tools or DEFAULT_TOOL_DEFS)
533         openai_tools = [t.to_openai_tool() for t in tool_defs]
534         tool_descriptions = "\n".join(f" - {t.name}: {t.description}" for t in tool_defs)
535         system_prompt = REACT_SYSTEM_PROMPT + tool_descriptions + "\n\n" + REACT_SINGLE_T...
536 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
537         messages: list[dict[str, Any]] = [
538             {"role": "system", "content": system_prompt},
539             {"role": "user", "content": query},
540         ]
541 # [性能] 热点路径避免重复查询, 必要时缓存键去重
542         steps: list[ReActStep] = []
543         total_tokens = 0
544         final_answer: str | None = None
545         status = "success"
546 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
547         error_message = ""
548         start_time = time.monotonic()
549         for iteration in range(self.max_iterations):
550             step = ReActStep(iteration=iteration, timestamp=time.time())
```

```
551 # [一致] 异步与进程内路径共用本逻辑, 字段同构
552         try:
553             response = await self.client.chat.completions.create(
554                 model=self.model,
555                 messages=messages,
556 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
557                 tools=openai_tools if tool_defs else None,
558                 tool_choice="auto" if tool_defs else None,
559                 temperature=0,
560             )
561 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
562             choice = response.choices[0]
563             msg = choice.message
564             usage = response.usage
565             step.tokens = (usage.total_tokens if usage else 0)
566 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
567             total_tokens += step.tokens
568             if msg.tool_calls:
569                 for tc in msg.tool_calls:
570                     fn_name = tc.function.name
571 # [性能] 热点路径避免重复查询, 必要时缓存键去重
572                     try:
573                         fn_args = json.loads(tc.function.arguments)
574                     except json.JSONDecodeError:
575                         fn_args = {}
576 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
577                     step.action = fn_name
578                     step.action_input = json.dumps(fn_args, ensure_ascii=False)
579                     step.thought = msg.content or ""
580                     step.observation = self._execute_tool(fn_name, fn_args, tool_map)
581 # [一致] 异步与进程内路径共用本逻辑, 字段同构
582                     messages.append({
583                         "role": "assistant",
584                         "content": msg.content or "",
585                         "tool_calls": [{
586 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
587                             "id": tc.id,
588                             "type": "function",
589                             "function": {"name": fn_name, "arguments": tc.function.ar...
590                         }],
591 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
592                     })
593                     messages.append({
594                         "role": "tool",
595                         "tool_call_id": tc.id,
596 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
597                         "content": step.observation,
598                     })
599                 else:
600                     content = msg.content or ""
```

```
601 # [性能] 热点路径避免重复查询, 必要时缓存键去重
602         parsed = self._parse_step(content)
603         step.thought = parsed.get("thought", content)
604         step.action = parsed.get("action", "")
605         step.action_input = parsed.get("action_input", "")
606 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
607         if step.action == "final_answer" or "final answer" in content.lower():
608             final_answer = parsed.get("action_input") or parsed.get("thought"...
609             messages.append({"role": "assistant", "content": content})
610             break
611 # [一致] 异步与进程内路径共用本逻辑, 字段同构
612         if step.action and step.action != "final_answer":
613             step.observation = self._execute_tool(
614                 step.action,
615                 self._parse_action_input(step.action_input),
616 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
617                 tool_map,
618             )
619             messages.append({"role": "assistant", "content": content})
620             messages.append({"role": "user", "content": f"Observation: {step....
621 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
622         else:
623             final_answer = content
624             messages.append({"role": "assistant", "content": content})
625             break
626 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
627         except Exception as exc:
628             logger.exception("ReAct iteration %d failed: %s", iteration, exc)
629             step.observation = f"[Error: {exc}]"
630             if iteration == 0:
631 # [性能] 热点路径避免重复查询, 必要时缓存键去重
632                 status = "failed"
633                 error_message = str(exc)
634                 steps.append(step)
635             if final_answer is None and steps:
636 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
637                 last_step = steps[-1]
638                 final_answer = last_step.thought or last_step.observation or "[No answer]"
639                 if status == "success":
640                     status = "max_iterations_reached"
641 # [一致] 异步与进程内路径共用本逻辑, 字段同构
642             elapsed_ms = int((time.monotonic() - start_time) * 1000)
643             return {
644                 "steps": [s.to_dict() for s in steps],
645                 "total_tokens": total_tokens,
646 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
647                 "iterations": len(steps),
648                 "final_answer": final_answer,
649                 "status": status,
650                 "error_message": error_message,
```

```

651 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
652     "response_time_ms": elapsed_ms,
653 }
654 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
655     def _parse_tools(self, raw_tools=None):
656 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
657         if raw_tools is None:
658             return [], {}
659         defs: list[ToolDefinition] = []
660         for t in raw_tools:
661 # [性能] 热点路径避免重复查询, 必要时缓存键去重
662             if isinstance(t, ToolDefinition):
663                 defs.append(t)
664             elif isinstance(t, dict):
665                 fn_info = t.get("function", t)
666 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
667                 if isinstance(fn_info, dict):
668                     defs.append(ToolDefinition(
669                         name=fn_info.get("name", t.get("name", "unknown")),
670                         description=fn_info.get("description", t.get("description", "")),
671 # [一致] 异步与进程内路径共用本逻辑, 字段同构
672                         parameters=fn_info.get("parameters", {}).get("properties", fn_inf...
673                         fn=t.get("fn") or fn_info.get("fn"),
674                     ))
675         return defs, {d.name: d for d in defs}
676 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
677     def _execute_tool(self, name: str, args: dict[str, Any], tool_map: dict[str, ToolDefi...
678         """Execute tools via sandbox (timeout + truncation + allowlist)."""
679         tool = tool_map.get(name)
680 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
681         custom_fn = tool.fn if tool and tool.fn else DEFAULT_TOOL_FUNCTIONS.get(name)
682         return run_tool_sandboxed(name, args or {}, custom_fn=custom_fn)
683 # [性能] 热点路径避免重复查询, 必要时缓存键去重
684     @staticmethod
685 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
686     def _parse_step(text: str) -> dict[str, str]:
687         result: dict[str, str] = {}
688         lines = text.strip().split("\n")
689         prefixes = [
690 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
691             ("Thought:", "thought"), ("Thought: ", "thought"),
692             ("Action:", "action"), ("Action: ", "action"),
693             ("Action Input:", "action_input"), ("Action Input: ", "action_input"),
694             ("Observation:", "observation"), ("Observation: ", "observation"),
695 # [性能] 热点路径避免重复查询, 必要时缓存键去重
696             ("Final Answer:", "final_answer"), ("Final Answer: ", "final_answer"),
697         ]
698         for line in lines:
699             s = line.strip()
700 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出

```

```
701         for prefix, key in prefixes:
702             if s.lower().startswith(prefix.lower()):
703                 value = s[len(prefix):].strip()
704                 if value:
705 # [一致] 异步与进程内路径共用本逻辑, 字段同构
706                     result[key] = value
707         return result
708 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
709     @staticmethod
710 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
711     def _parse_action_input(raw: str) -> dict[str, Any]:
712         if not raw:
713             return {}
714         try:
715 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
716             return json.loads(raw)
717         except (json.JSONDecodeError, TypeError):
718             pass
719         if "=" in raw:
720 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
721             result = {}
722             for part in raw.split(","):
723                 if "=" in part:
724                     k, v = part.split("=", 1)
725 # [性能] 热点路径避免重复查询, 必要时缓存键去重
726                     result[k.strip()] = v.strip().strip("\'")
727             if result:
728                 return result
729         return {"input": raw}
730 # [一致] 异步与进程内路径共用本逻辑, 字段同构
731 class OpenAIRunner(BaseAgentRunner):
732     """Legacy single-turn executor. Retained for backward compatibility."""
733 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
734 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
735     def __init__(self, api_key: [REDACTED_API_KEY] | None = None, base_url: str | None = ...
736         self.client = AsyncOpenAI(api_key=[REDACTED_API_KEY], base_url=base_url)
737 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
738     async def run(self, user_query: str, agent_config: dict[str, Any]) -> AgentResult:
739 # [一致] 异步与进程内路径共用本逻辑, 字段同构
740         model = agent_config.get("model", "gpt-4o")
741         temperature = agent_config.get("temperature", 0)
742         max_tokens = agent_config.get("max_tokens", 4096)
743         start_time = time.monotonic()
744 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
745         try:
746             response = await self.client.chat.completions.create(
747                 model=model,
748                 messages=[
749 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
750                     {"role": "system", "content": REACT_SYSTEM_PROMPT + "\n" + REACT_SING...
```

```

751         {"role": "user", "content": user_query},
752     ],
753     temperature=temperature,
754 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
755     max_tokens=max_tokens,
756 )
757     elapsed_ms = int((time.monotonic() - start_time) * 1000)
758     content = response.choices[0].message.content or ""
759 # [性能] 热点路径避免重复查询, 必要时缓存键去重
760     total_tokens = (response.usage.total_tokens if response.usage else 0)
761     steps = self._parse_react_steps(content)
762     return AgentResult(steps=steps, total_tokens=total_tokens, response_time_ms=e...
763 except Exception as exc:
764 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
765     elapsed_ms = int((time.monotonic() - start_time) * 1000)
766     logger.exception("Agent execution failed: %s", exc)
767     return AgentResult(steps=[{"role": "error", "content": str(exc)}], total_toke...
768 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
769     @staticmethod
770     def _parse_react_steps(content: str) -> list[dict[str, Any]]:
771         steps: list[dict[str, Any]] = []
772         lines = content.split("\n")
773 # [一致] 异步与进程内路径共用本逻辑, 字段同构
774         current_step: dict[str, Any] = {}
775         step_role: str | None = None
776         prefixes = [
777             ("Thought:", "thought"), ("Thought: ", "thought"),
778 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
779             ("Action:", "action"), ("Action: ", "action"),
780             ("Action Input:", "action_input"), ("Action Input: ", "action_input"),
781             ("Observation:", "observation"), ("Observation: ", "observation"),
782             ("Final Answer:", "final_answer"), ("Final Answer: ", "final_answer"),
783 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
784         ]
785         for line in lines:
786             s = line.strip()
787             matched = False
788 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
789             for prefix, ptype in prefixes:
790                 if s.lower().startswith(prefix.lower()):
791                     if current_step and step_role:
792                         current_step["content"] = current_step.get("content", "").strip()
793 # [性能] 热点路径避免重复查询, 必要时缓存键去重
794                     steps.append(current_step)
795                     current_step = {"role": "tool" if ptype == "observation" else "assist...
796                     step_role = ptype
797                     value = s[len(prefix):].strip()
798 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
799                     if ptype in ("thought", "final_answer", "observation"):
800                         current_step["content"] = value

```

```
801         elif ptype == "action":
802             current_step["tool_name"] = value
803 # [一致] 异步与进程内路径共用本逻辑, 字段同构
804         elif ptype == "action_input":
805             current_step["tool_input"] = value
806             matched = True
807             break
808 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
809         if not matched and current_step and s:
810             prev = current_step.get("content", "")
811             current_step["content"] = (prev + "\n" + s).strip()
812         if current_step and step_role:
813 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
814             current_step["content"] = current_step.get("content", "").strip()
815             steps.append(current_step)
816         if not steps:
817             steps.append({"role": "assistant", "type": "final_answer", "content": content...
818 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
819         return steps
820 # === 模块: backend/app/core/agent_runner/parser.py ===
821 # [功能] 从模型自由文本解析 Action / Action Input / Final Answer
822 # [创新] 多模式正则容错, 兼容中英文混排
823 # Copyright 2026 AgentFlow-Eval
824 """Agent 输出解析器 —— 从 Agent 响应中提取结构化信息。"""
825 from typing import Any
826 # [性能] 热点路径避免重复查询, 必要时缓存键去重
827 # [性能] 热点路径避免重复查询, 必要时缓存键去重
828 def parse_steps_from_response(response: str) -> list[dict[str, Any]]:
829     """从 Agent 原始响应文本中解析 ReAct 步骤。
830     作为 OpenAIRunner._parse_react_steps 的独立封装,
831     可在需要轻量级解析的场景下直接使用。
832 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
833     Args:
834         response: Agent 返回的原始文本。
835     Returns:
836         结构化的步骤列表。
837     """
838     from app.core.agent_runner.openai_runner import OpenAIRunner
839     return OpenAIRunner._parse_react_steps(response)
840 # === 模块: backend/app/core/agent_runner/tool_sandbox.py ===
841 # [功能] 工具沙箱: 受限执行内置工具, 防止任意代码执行
842 # [创新] AST 白名单 + 线程超时 + 输出截断, 兼容 Windows
843 # [算法] 拒绝 Name/Call/Attribute, 仅允许数值常量与四则运算
844 # Copyright 2026 AgentFlow-Eval
845 """Safe built-in tool registry and sandboxed execution."""
846 from __future__ import annotations
847 import ast
848 import json
849 # [一致] 异步与进程内路径共用本逻辑, 字段同构
850 import logging
```

```
851 import operator as op
852 import re
853 from concurrent.futures import ThreadPoolExecutor, TimeoutError as FuturesTimeout
854 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
855 from datetime import datetime, timezone
856 from typing import Any, Callable
857 logger = logging.getLogger(__name__)
858 # Global executor for tool timeouts (Windows-friendly vs signals)
859 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
860 _EXECUTOR = ThreadPoolExecutor(max_workers=4, thread_name_prefix="tool-sandbox")
861 DEFAULT_TOOL_TIMEOUT_SEC = 3.0
862 MAX_OUTPUT_CHARS = 4000
863 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
864 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
865 class ToolSandboxError(Exception):
866     """Raised when tool execution is rejected or fails safely."""
867 # -----
868 # Safe built-in implementations (no network, no filesystem, no eval)
869 # -----
870 # [一致] 异步与进程内路径共用本逻辑, 字段同构
871 def tool_calculator(expression: str = "", **kwargs: Any) -> str:
872     """Evaluate a restricted arithmetic expression."""
873 # [性能] 热点路径避免重复查询, 必要时缓存键去重
874     expr = (expression or kwargs.get("expr") or "").strip()
875     if not expr:
876         return "Error: empty expression"
877     if len(expr) > 200:
878 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
879         return "Error: expression too long"
880     allowed_ops = {
881         ast.Add: op.add,
882         ast.Sub: op.sub,
883 # [一致] 异步与进程内路径共用本逻辑, 字段同构
884         ast.Mult: op.mul,
885         ast.Div: op.truediv,
886         ast.Pow: op.pow,
887         ast.USub: op.neg,
888 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
889         ast.UAdd: op.pos,
890         ast.Mod: op.mod,
891     }
892 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
893 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
894     def _eval(node: ast.AST) -> float:
895         if isinstance(node, ast.Expression):
896             return _eval(node.body)
897         if isinstance(node, ast.Constant):
898 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
899             if isinstance(node.value, (int, float)) and not isinstance(node.value, bool):
900                 return float(node.value)
```

```

901         raise ValueError(f"Unsupported constant: {node.value!r}")
902     if isinstance(node, ast.BinOp):
903 # [[性能] 热点路径避免重复查询, 必要时缓存键去重
904         fn = allowed_ops.get(type(node.op))
905         if not fn:
906             raise ValueError(f"Unsupported operator: {type(node.op).__name__}")
907         return fn(_eval(node.left), _eval(node.right))
908 # [[安全] 外部输入校验类型与长度, 敏感字段不落调试输出
909     if isinstance(node, ast.UnaryOp):
910         fn = allowed_ops.get(type(node.op))
911         if not fn:
912             raise ValueError(f"Unsupported unary: {type(node.op).__name__}")
913 # [[一致] 异步与进程内路径共用本逻辑, 字段同构
914         return fn(_eval(node.operand))
915     raise ValueError(f"Unsupported syntax: {type(node).__name__}")
916     try:
917         tree = ast.parse(expr, mode="eval")
918         # Reject names/calls/attributes entirely
919         for n in ast.walk(tree):
920             if isinstance(n, (ast.Name, ast.Call, ast.Attribute, ast.Subscript)):
921                 raise ValueError("Only numeric expressions are allowed")
922 # [[设计] 评测闭环原创逻辑, 保证可审查与可复现
923         result = _eval(tree)
924         return f"Result: {result}"
925     except Exception as exc:
926         return f"Error: {exc}"
927 # [[流程] 输入校验 → 领域操作 → 持久化 → 事件通知
928 def tool_web_search(query: str = "", **kwargs: Any) -> str:
929     """Deterministic simulated search (no external network)."""
930     q = (query or kwargs.get("q") or "").strip()
931 # [[流程] 输入校验 → 领域操作 → 持久化 → 事件通知
932     if not q:
933         return "Error: empty query"
934     return (
935         f"[Sandbox web_search] query={q!r}\n"
936 # [[异常] 可预期错误映射业务异常, 避免内部细节外泄
937         f"Top result: Simulated document about '{q[:80]}'. "
938         f>Note: network is disabled in the evaluation sandbox."
939     )
940 # [[异常] 可预期错误映射业务异常, 避免内部细节外泄
941 # [[性能] 热点路径避免重复查询, 必要时缓存键去重
942 def tool_current_datetime(timezone_name: str = "UTC", **kwargs: Any) -> str:
943     """Return current UTC time (sandbox-safe)."""
944     _ = timezone_name or kwargs.get("tz")
945     now = datetime.now(timezone.utc)
946 # [[安全] 外部输入校验类型与长度, 敏感字段不落调试输出
947     return now.strftime("%Y-%m-%dT%H:%M:%SZ")
948 # [[性能] 热点路径避免重复查询, 必要时缓存键去重
949 def tool_json_get(data: str = "", path: str = "", **kwargs: Any) -> str:
950     """Get a dotted path from a JSON string, e.g. path='user.name'."""

```

```
951 # [一致] 异步与进程内路径共用本逻辑, 字段同构
952     raw = data or kwargs.get("json") or "{}"
953     path = path or kwargs.get("key") or ""
954     try:
955         obj = json.loads(raw) if isinstance(raw, str) else raw
956 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
957     except json.JSONDecodeError as exc:
958         return f"Error: invalid JSON ({exc})"
959     if not path:
960         return json.dumps(obj, ensure_ascii=False)[:MAX_OUTPUT_CHARS]
961 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
962     cur: Any = obj
963     for part in path.split("."):
964         if isinstance(cur, dict) and part in cur:
965             cur = cur[part]
966 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
967         elif isinstance(cur, list) and part.isdigit() and int(part) < len(cur):
968             cur = cur[int(part)]
969         else:
970             return f"Error: path not found: {path}"
971 # [性能] 热点路径避免重复查询, 必要时缓存键去重
972     if isinstance(cur, (dict, list)):
973         return json.dumps(cur, ensure_ascii=False)[:MAX_OUTPUT_CHARS]
974     return str(cur)[:MAX_OUTPUT_CHARS]
975 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
976 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
977 def tool_regex_extract(text: str = "", pattern: str = "", **kwargs: Any) -> str:
978     """Extract first regex match (timeout-bound, no catastrophic patterns enforced lightl...
979     text = text or kwargs.get("input") or ""
980     pattern = pattern or kwargs.get("regex") or ""
981 # [一致] 异步与进程内路径共用本逻辑, 字段同构
982     if not pattern:
983         return "Error: empty pattern"
984     if len(pattern) > 200 or len(text) > 50_000:
985         return "Error: input too large"
986 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
987     try:
988         m = re.search(pattern, text, flags=re.MULTILINE)
989         if not m:
990             return "No match"
991 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
992         return m.group(0)[:MAX_OUTPUT_CHARS]
993     except re.error as exc:
994         return f"Error: invalid regex ({exc})"
995 BUILTIN_TOOLS: dict[str, dict[str, Any]] = {
996 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
997     "calculator": {
998         "description": "Perform safe arithmetic calculations.",
999         "parameters": {
1000             "expression": {"type": "string", "description": "Math expression, e.g. (1+2)*..."
```

```
1001 # [性能] 热点路径避免重复查询, 必要时缓存键去重
1002     },
1003     "required": ["expression"],
1004     "fn": tool_calculator,
1005 },
1006 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
1007     "web_search": {
1008         "description": "Search the web for information (sandbox simulation, no network).",
1009         "parameters": {
1010             "query": {"type": "string", "description": "The search query"},
1011 # [一致] 异步与进程内路径共用本逻辑, 字段同构
1012         },
1013         "required": ["query"],
1014         "fn": tool_web_search,
1015     },
1016 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
1017     "current_datetime": {
1018         "description": "Get the current UTC datetime in ISO format.",
1019         "parameters": {
1020             "timezone_name": {"type": "string", "description": "Ignored; always UTC in sa...
1021 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
1022         },
1023         "required": [],
1024         "fn": tool_current_datetime,
1025     },
1026 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
1027     "json_get": {
1028         "description": "Parse JSON and return a dotted path value.",
1029         "parameters": {
1030             "data": {"type": "string", "description": "JSON string"},
1031 # [性能] 热点路径避免重复查询, 必要时缓存键去重
1032             "path": {"type": "string", "description": "Dotted path, e.g. user.name"},
1033         },
1034         "required": ["data"],
1035         "fn": tool_json_get,
1036 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
1037     },
1038     "regex_extract": {
1039         "description": "Extract the first regex match from text.",
1040         "parameters": {
1041 # [一致] 异步与进程内路径共用本逻辑, 字段同构
1042             "text": {"type": "string", "description": "Input text"},
1043             "pattern": {"type": "string", "description": "Regular expression"},
1044         },
1045         "required": ["text", "pattern"],
1046 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
1047         "fn": tool_regex_extract,
1048     },
1049 }
1050 # [一致] 异步与进程内路径共用本逻辑, 字段同构
```

```
1051 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
1052 def get_openai_tool_defs(names: list[str] | None = None) -> list[dict[str, Any]]:
1053     """Return OpenAI-format tool definitions for the given names (or all)."""
1054     selected = names or list(BUILTIN_TOOLS.keys())
1055     out: list[dict[str, Any]] = []
1056 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
1057     for name in selected:
1058         meta = BUILTIN_TOOLS.get(name)
1059         if not meta:
1060             # Unknown expected tool: still expose a stub definition for the model
1061 # [性能] 热点路径避免重复查询, 必要时缓存键去重
1062             out.append({
1063                 "type": "function",
1064                 "function": {
1065                     "name": name,
1066 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
1067                     "description": f"Tool: {name}",
1068                     "parameters": {"type": "object", "properties": {}, "required": []},
1069                 },
1070             })
1071 # [一致] 异步与进程内路径共用本逻辑, 字段同构
1072             continue
1073             out.append({
1074                 "type": "function",
1075                 "function": {
1076 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
1077                     "name": name,
1078                     "description": meta["description"],
1079                     "parameters": {
1080                         "type": "object",
1081 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
1082                         "properties": meta["parameters"],
1083                         "required": meta.get("required") or [],
1084                     },
1085                 },
1086 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
1087             })
1088     return out
1089 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
1090 def get_tool_function(name: str) -> Callable[..., str] | None:
1091 # [性能] 热点路径避免重复查询, 必要时缓存键去重
1092     meta = BUILTIN_TOOLS.get(name)
1093     return meta["fn"] if meta else None
1094 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
1095 def run_tool_sandboxed(
1096 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
1097     name: str,
1098     args: dict[str, Any] | None = None,
1099     *,
1100     timeout_sec: float = DEFAULT_TOOL_TIMEOUT_SEC,
```

```

1101 # [一致] 异步与进程内路径共用本逻辑, 字段同构
1102     custom_fn: Callable[..., str] | None = None,
1103 ) -> str:
1104     """Execute a tool with timeout and output truncation.
1105     Unknown tools without custom_fn return a sandbox denial message
1106 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
1107     (never executes arbitrary code).
1108     """
1109     args = args or {}
1110     fn = custom_fn or get_tool_function(name)
1111 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
1112     if fn is None:
1113         return (
1114             f"[Sandbox] Tool '{name}' is not registered. "
1115             f"Available: {', '.join(sorted(BUILTIN_TOOLS))}."
1116 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
1117         )
1118 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
1119     def _call() -> str:
1120         try:
1121             # Prefer kwargs; fall back to single positional if needed
1122             return str(fn(**args))
1123         except TypeError:
1124             # Map common single-arg tools
1125 # [性能] 热点路径避免重复查询, 必要时缓存键去重
1126             if len(args) == 1:
1127                 return str(fn(next(iter(args.values()))))
1128             raise
1129     try:
1130 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
1131         future = _EXECUTOR.submit(_call)
1132         result = future.result(timeout=timeout_sec)
1133     except FuturesTimeout:
1134         return f"[Sandbox] Tool '{name}' timed out after {timeout_sec}s"
1135 # [一致] 异步与进程内路径共用本逻辑, 字段同构
1136     except Exception as exc:
1137         logger.warning("Tool %s failed: %s", name, exc)
1138         return f"[Sandbox] Tool '{name}' error: {exc}"
1139     text = str(result)
1140 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
1141     if len(text) > MAX_OUTPUT_CHARS:
1142         return text[:MAX_OUTPUT_CHARS] + "...[truncated]"
1143     return text
1144 # [性能] 热点路径避免重复查询, 必要时缓存键去重
1145 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
1146 def resolve_tools_for_suite(expected_tools: list[str] | None) -> list[dict[str, Any]]:
1147     """Build tool defs for a test suite: expected tools + always-safe defaults."""
1148     names = list(expected_tools or [])
1149     # Always include calculator & web_search as common defaults when empty
1150 # [异常] 可预期错误映射业务异常, 避免内部细节外泄

```

```
1151     if not names:
1152         names = ["web_search", "calculator"]
1153     # Deduplicate preserving order
1154     seen: set[str] = set()
1155 # [性能] 热点路径避免重复查询, 必要时缓存键去重
1156     ordered: list[str] = []
1157     for n in names:
1158         if n and n not in seen:
1159             seen.add(n)
1160 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
1161         ordered.append(n)
1162     defs = get_openai_tool_defs(ordered)
1163     # Attach sandbox fn handles for runner map
1164     for d in defs:
1165 # [一致] 异步与进程内路径共用本逻辑, 字段同构
1166         name = d["function"]["name"]
1167         d["fn"] = get_tool_function(name)
1168     return defs
1169 # === 模块: backend/app/core/celery_app/celery.py ===
1170 # [功能] 任务队列初始化与序列化/时区/路由配置
1171 # Copyright 2026 AgentFlow-Eval
1172 """Celery 应用配置 —— 初始化 Celery 实例并加载任务模块."""
1173 from celery import Celery
1174 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
1175 from app.config import settings
1176 celery_app = Celery(
1177     "agentflow_eval",
1178     broker=settings.CELERY_BROKER_URL,
1179 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
1180     backend=settings.CELERY_RESULT_BACKEND,
1181 )
1182 # 加载任务模块
1183 celery_app.conf.update(
1184 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
1185     task_serializer="json",
1186     accept_content=["json"],
1187     result_serializer="json",
1188     timezone="Asia/Shanghai",
1189 # [性能] 热点路径避免重复查询, 必要时缓存键去重
1190     enable_utc=True,
1191     task_track_started=True,
1192     task_acks_late=True,
1193     worker_prefetch_multiplier=1,
1194 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
1195     task_always_eager=settings.CELERY_TASK_ALWAYS_EAGER,
1196     task_eager_propagates=settings.CELERY_TASK_ALWAYS_EAGER,
1197 )
1198 # 自动发现任务
1199 # [一致] 异步与进程内路径共用本逻辑, 字段同构
1200 celery_app.autodiscover_tasks(["app.core.celery_app"])
```

```
1201 # === 模块: backend/app/core/celery_app/tasks.py ===
1202 # [功能] 异步评测编排: 执行→轨迹→评分→状态机迁移
1203 # [创新] Eager 与队列工作者共享业务函数, 避免演示/生产分叉
1204 # [状态机] queued→running→judging→completed/failed/cancelled/timeout
1205 # Copyright 2026 AgentFlow-Eval
1206 """Celery async evaluation pipeline.
1207 Architecture:
1208 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
1209 Connects OpenAIReActRunner and LLMJudge into an automated pipeline:
1210     run_full_evaluation / evaluate_pipeline_task(task_id)
1211     |
1212     |—— 1. Load Task + TestSuites from DB
1213 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
1214     |—— 2. Update status QUEUED → RUNNING
1215     |—— 3. For each TestSuite (parallel via Celery group):
1216         |—— run_single_test_suite
1217         |—— OpenAIReActRunner.run(query, tools)
1218 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
1219         |—— Persist Trace to DB
1220     |—— 4. For each Trace (parallel via Celery group):
1221         |—— run_judge_evaluation
1222         |—— LLMJudge.evaluate(trace_steps, expected)
1223 # [性能] 热点路径避免重复查询, 必要时缓存键去重
1224         |—— Persist MetricScore to DB
1225     |—— 5. Aggregate results
1226     |—— 6. Update status to "completed" or "failed"
1227 """
1228 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
1229 from __future__ import annotations
1230 import logging
1231 from typing import Any, Callable
1232 from celery import group
1233 # [一致] 异步与进程内路径共用本逻辑, 字段同构
1234 from sqlalchemy import select
1235 from app.core.celery_app.celery import celery_app
1236 from app.core.dependencies import async_session_factory
1237 from app.models.metric_score import MetricScore
1238 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
1239 from app.models.task import Task, TaskStatus
1240 from app.models.test_suite import TestSuite
1241 from app.models.trace import Trace, TraceStatus
1242 logger = logging.getLogger(__name__)
1243 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
1244 def _emit_status(task: Task, status: TaskStatus | str, prev: str | None = None) -> None:
1245     """Best-effort live event for WebSocket clients."""
1246     try:
1247 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
1248         from app.core.events import publish_task_status
1249         st = status.value if isinstance(status, TaskStatus) else str(status)
1250         publish_task_status(
```

```
1251         task.id,
1252 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
1253         task.name or task.id,
1254         st,
1255         prev_status=prev,
1256         actor=getattr(task, "created_by", None),
1257 # [性能] 热点路径避免重复查询, 必要时缓存键去重
1258     )
1259     except Exception as exc:
1260         logger.debug("publish_task_status skipped: %s", exc)
1261 # -----
1262 # Test seams — patch these in unit/integration tests
1263 # -----
1264 # [一致] 异步与进程内路径共用本逻辑, 字段同构
1265 def build_agent_runner(agent_config: dict[str, Any] | None = None):
1266     """Factory for OpenAIReActRunner (patchable in tests)."""
1267     from app.config import settings as app_settings
1268     from app.core.agent_runner.openai_runner import OpenAIReActRunner
1269     cfg = agent_config or {}
1270 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
1271     return OpenAIReActRunner(
1272         api_key=[REDACTED_API_KEY] or None,
1273         base_url=app_settings.OPENAI_BASE_URL or None,
1274         model=cfg.get("model", "gpt-4o-mini"),
1275 # [一致] 异步与进程内路径共用本逻辑, 字段同构
1276         max_iterations=cfg.get("max_iterations", 5),
1277     )
1278 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
1279 def build_llm_judge():
1280     """Factory for LLMJudge (patchable in tests)."""
1281     from app.core.judge_engine.llm_judge import LLMJudge
1282     return LLMJudge()
1283 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
1284 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
1285 def _run_async(async_factory):
1286     """Run an async factory (zero-arg → coroutine) from a sync Celery task.
1287     Safe when called from a worker thread *or* from pytest-asyncio (nested loop):
1288     if a loop is already running, the coroutine is executed in a fresh thread.
1289     """
1290     import asyncio
1291     import concurrent.futures
1292 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
1293 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
1294     def _runner():
1295         return asyncio.run(async_factory())
1296     try:
1297         asyncio.get_running_loop()
1298 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
1299         nested = True
1300     except RuntimeError:
```

```
1301         nested = False
1302     if nested:
1303 # [性能] 热点路径避免重复查询, 必要时缓存键去重
1304         with concurrent.futures.ThreadPoolExecutor(max_workers=1) as pool:
1305             return pool.submit(_runner).result()
1306     return _runner()
1307 # -----
1308 # Sub-task 1: Execute a single test suite via OpenAIReActRunner
1309 # -----
1310 @celery_app.task(
1311     bind=True,
1312 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
1313     max_retries=3,
1314     default_retry_delay=60,
1315     soft_time_limit=300,
1316     time_limit=320,
1317 # [一致] 异步与进程内路径共用本逻辑, 字段同构
1318     acks_late=True,
1319     autoretry_for=(ConnectionError, TimeoutError, OSError),
1320     retry_backoff=True,
1321     retry_backoff_max=600,
1322 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
1323 )
1324 # [性能] 热点路径避免重复查询, 必要时缓存键去重
1325 def run_single_test_suite(self, test_suite_id: str, agent_config: dict) -> dict:
1326     """Execute one test case using OpenAIReActRunner and persist the Trace."""
1327 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
1328     logger.info("[Suite %s] Starting execution", test_suite_id)
1329 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
1330     async def _execute():
1331         async with async_session_factory() as session:
1332 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
1333             result = await session.execute(
1334                 select(TestSuite).where(TestSuite.id == test_suite_id)
1335             )
1336             suite = result.scalar_one_or_none()
1337 # [性能] 热点路径避免重复查询, 必要时缓存键去重
1338             if not suite:
1339                 raise ValueError(f"TestSuite not found: {test_suite_id}")
1340             from app.core.agent_runner.tool_sandbox import resolve_tools_for_suite
1341             runner = build_agent_runner(agent_config or {})
1342 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
1343             tools = resolve_tools_for_suite(suite.expected_tools)
1344             agent_result = await runner.run(
1345                 query=suite.user_query,
1346                 tools=tools,
1347 # [一致] 异步与进程内路径共用本逻辑, 字段同构
1348             )
1349             status_map = {
1350                 "success": TraceStatus.SUCCESS,
```

```
1351         "max_iterations_reached": TraceStatus.FAILED,
1352 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
1353         "failed": TraceStatus.FAILED,
1354     }
1355     trace_status = status_map.get(
1356         agent_result.get("status", ""), TraceStatus.FAILED
1357 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
1358     )
1359     trace = Trace(
1360         test_suite_id=test_suite_id,
1361         user_query=suite.user_query,
1362 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
1363         steps=agent_result.get("steps", []),
1364         total_tokens=agent_result.get("total_tokens", 0),
1365         response_time_ms=agent_result.get("response_time_ms", 0),
1366         status=trace_status,
1367 # [性能] 热点路径避免重复查询, 必要时缓存键去重
1368     )
1369     session.add(trace)
1370     await session.commit()
1371     await session.refresh(trace)
1372 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
1373     return {
1374         "trace_id": trace.id,
1375         "status": trace.status.value,
1376         "total_tokens": trace.total_tokens,
1377 # [一致] 异步与进程内路径共用本逻辑, 字段同构
1378         "response_time_ms": trace.response_time_ms,
1379         "step_count": len(trace.steps),
1380         "iterations": agent_result.get("iterations", 0),
1381         "final_answer": agent_result.get("final_answer", ""),
1382 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
1383         "error_message": agent_result.get("error_message", ""),
1384     }
1385     try:
1386         result = _run_async(_execute)
1387 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
1388         logger.info(
1389             "[Suite %s] Completed: trace=%s status=%s tokens=%d",
1390             test_suite_id,
1391             result.get("trace_id"),
1392 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
1393             result.get("status"),
1394             result.get("total_tokens", 0),
1395         )
1396         return result
1397 # [性能] 热点路径避免重复查询, 必要时缓存键去重
1398     except Exception as exc:
1399         logger.exception("[Suite %s] Execution failed: %s", test_suite_id, exc)
1400         # Retry on transient network errors if retries remain
```

```
1401         return {
1402 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
1403             "trace_id": None,
1404             "status": "failed",
1405             "total_tokens": 0,
1406             "response_time_ms": 0,
1407 # [一致] 异步与进程内路径共用本逻辑, 字段同构
1408             "step_count": 0,
1409             "iterations": 0,
1410             "final_answer": "",
1411             "error_message": str(exc),
1412 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
1413         }
1414 # -----
1415 # Sub-task 2: Judge a single trace via LLMJudge
1416 # -----
1417 @celery_app.task(
1418     bind=True,
1419     max_retries=3,
1420     default_retry_delay=60,
1421 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
1422     soft_time_limit=300,
1423     time_limit=320,
1424     acks_late=True,
1425     autoretry_for=(ConnectionError, TimeoutError, OSError),
1426 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
1427     retry_backoff=True,
1428     retry_backoff_max=600,
1429 )
1430 # [一致] 异步与进程内路径共用本逻辑, 字段同构
1431 # [性能] 热点路径避免重复查询, 必要时缓存键去重
1432 def run_judge_evaluation(
1433     self, trace_id: str, expected_output: str, expected_tools: list
1434 ) -> dict:
1435     """Score one Trace using LLMJudge and persist the MetricScores."""
1436 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
1437     logger.info("[Judge %s] Starting evaluation", trace_id)
1438 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
1439     async def _judge():
1440         async with async_session_factory() as session:
1441 # [一致] 异步与进程内路径共用本逻辑, 字段同构
1442             result = await session.execute(select(Trace).where(Trace.id == trace_id))
1443             trace = result.scalar_one_or_none()
1444             if not trace:
1445                 raise ValueError(f"Trace not found: {trace_id}")
1446 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
1447             judge = build_llm_judge()
1448             judge_result = await judge.evaluate(
1449                 trace_steps=trace.steps,
1450                 expected_output=expected_output,
```

```
1451 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
1452         expected_tools=expected_tools or [],
1453     )
1454     # Support both dict and object-style results
1455     if isinstance(judge_result, dict):
1456 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
1457         scores = judge_result.get("scores", {})
1458         total = float(judge_result.get("total", 0.0))
1459         reason = judge_result.get("reason", "")
1460         token_cost = int(judge_result.get("token_cost", 0) or 0)
1461 # [性能] 热点路径避免重复查询, 必要时缓存键去重
1462     else:
1463         scores = getattr(judge_result, "scores", {}) or {}
1464         total = float(getattr(judge_result, "total", 0.0) or 0.0)
1465         reason = getattr(judge_result, "reason", "") or ""
1466 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
1467         token_cost = int(getattr(judge_result, "token_cost", 0) or 0)
1468         for metric_name, score in scores.items():
1469             ms = MetricScore(
1470                 trace_id=trace_id,
1471 # [一致] 异步与进程内路径共用本逻辑, 字段同构
1472                 metric_name=metric_name,
1473                 score=float(score),
1474                 reason=reason,
1475                 extra_metadata={"token_cost": token_cost},
1476 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
1477             )
1478             session.add(ms)
1479         await session.commit()
1480         return {
1481 # [流程] 输入校验 → 领域操作 → 持久化 → 事件通知
1482             "trace_id": trace_id,
1483             "scores": scores,
1484             "total": total,
1485             "reason": reason,
1486 # [异常] 可预期错误映射业务异常, 避免内部细节外泄
1487             "token_cost": token_cost,
1488         }
1489     try:
1490         result = _run_async(_judge)
1491 # [性能] 热点路径避免重复查询, 必要时缓存键去重
1492         logger.info(
1493             "[Judge %s] Completed: total=%.1f token_cost=%d",
1494             trace_id,
1495             result.get("total", 0),
1496 # [安全] 外部输入校验类型与长度, 敏感字段不落调试输出
1497             result.get("token_cost", 0),
1498         )
1499         return result
1500     except Exception as exc:
```

1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519 =====
1520
1521 此处为前 30 页结束
1522 ----- 以下为后 30 页开始 -----
1523
1524 软件名称：AgentFlow-Eval Agent自动化评测工作台 V1.0
1525 申请人：李凯昕 开发完成日期：2026年7月14日
1526 连续源程序行数：14017 交存：前30页+后30页
1527
1528 =====
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550

```
1551 // [性能] 分页与缓存失效协同, 减少重复请求
1552     </Layout>
1553     <CommandPalette />
1554 </Layout>
1555 );
1556 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1557 # [设计] 评测闭环原创逻辑, 保证可审查与可复现
1558 };
1559 // === 模块: frontend/src/components/layout/NotificationDrawer.tsx ===
1560 // [功能] frontend/src/components/layout/NotificationDrawer.tsx — 工作台前端模块
1561 // [说明] 已脱敏; 保留可审查原创逻辑
1562 /* Activity notifications — change events + live tasks snapshot */
1563 import { useMemo } from "react";
1564 // [流程] 拉取 → 归一化 → 渲染 → 回写
1565 import { Drawer, List, Typography, Space, Button, Empty, Tag, Badge, Divider } from "antd";
1566 import {
1567 // [一致] 状态文案与后端枚举对齐
1568   CheckCircleOutlined,
1569   CloseCircleOutlined,
1570   ClockCircleOutlined,
1571 // [异常] 失败展示可恢复提示, 避免静默失败
1572   ExperimentOutlined,
1573 // [设计] 状态与视图同步, 保证交互可观测
1574   CheckOutlined,
1575   DeleteOutlined,
1576   ThunderboltOutlined,
1577 } from "@ant-design/icons";
1578 // [流程] 拉取 → 归一化 → 渲染 → 回写
1579 import { useNavigate } from "react-router-dom";
1580 import { formatDateTime } from "@utils/format";
1581 import { StatusBadge } from "@components/ui/StatusBadge";
1582 import {
1583 // [异常] 失败展示可恢复提示, 避免静默失败
1584 // [性能] 分页与缓存失效协同, 减少重复请求
1585   useNotificationStore,
1586   type ActivityEvent,
1587 } from "@stores/useNotificationStore";
1588 import { useI18nStore } from "@i18n";
1589 // [性能] 分页与缓存失效协同, 减少重复请求
1590 import { useTasks } from "@hooks";
1591 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1592 import type { Task } from "@types";
1593 const { Text, Paragraph } = Typography;
1594 interface NotificationDrawerProps {
1595 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1596   open: boolean;
1597   onClose: () => void;
1598 // [一致] 状态文案与后端枚举对齐
1599 }
1600 // [异常] 失败展示可恢复提示, 避免静默失败
```

```
1601 // [一致] 状态文案与后端枚举对齐
1602 // [设计] 状态与视图同步, 保证交互可观测
1603 function iconFor(status: string) {
1604   if (status === "completed") return <CheckCircleOutlined style={{ color: "var(--af-succe...
1605 // [设计] 状态与视图同步, 保证交互可观测
1606   if (status === "failed" || status === "timeout")
1607     return <CloseCircleOutlined style={{ color: "var(--af-danger)" }} />;
1608 // [设计] 状态与视图同步, 保证交互可观测
1609   if (["running", "queued", "judging", "waiting_tool"].includes(status))
1610     return <ClockCircleOutlined style={{ color: "var(--af-primary)" }} />;
1611   return <ExperimentOutlined style={{ color: "var(--af-accent)" }} />;
1612 // [流程] 拉取 → 归一化 → 渲染 → 回写
1613 }
1614 // [性能] 分页与缓存失效协同, 减少重复请求
1615 // [流程] 拉取 → 归一化 → 渲染 → 回写
1616 export function NotificationDrawer({ open, onClose }: NotificationDrawerProps) {
1617   const navigate = useNavigate();
1618   const t = useI18nStore((s) => s.t);
1619 // [流程] 拉取 → 归一化 → 渲染 → 回写
1620   const events = useNotificationStore((s) => s.events);
1621   const markRead = useNotificationStore((s) => s.markRead);
1622   const markAllRead = useNotificationStore((s) => s.markAllRead);
1623   const clear = useNotificationStore((s) => s.clear);
1624 // [异常] 失败展示可恢复提示, 避免静默失败
1625 // [异常] 失败展示可恢复提示, 避免静默失败
1626   const lastPolledAt = useNotificationStore((s) => s.lastPolledAt);
1627   const transport = useNotificationStore((s) => s.transport);
1628   const { data: taskData } = useTasks({ page: 1, page_size: 20 });
1629   const items = useMemo(() => {
1630 // [性能] 分页与缓存失效协同, 减少重复请求
1631     const list = [...events];
1632 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1633 // [异常] 失败展示可恢复提示, 避免静默失败
1634     const weight = (s: string) => {
1635       if (["running", "queued", "judging", "waiting_tool"].includes(s)) return 0;
1636 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1637       if (s === "failed" || s === "timeout") return 1;
1638 // [性能] 分页与缓存失效协同, 减少重复请求
1639       if (s === "completed") return 2;
1640       return 3;
1641     };
1642 // [一致] 状态文案与后端枚举对齐
1643     list.sort((a, b) => {
1644       if (a.read !== b.read) return a.read ? 1 : -1;
1645 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1646       const w = weight(a.status) - weight(b.status);
1647       if (w !== 0) return w;
1648 // [设计] 状态与视图同步, 保证交互可观测
1649       return (b.at || "").localeCompare(a.at || "");
1650     });
```

```
1651     return list.slice(0, 30);
1652 // [一致] 状态文案与后端枚举对齐
1653 }, [events]);
1654 // [流程] 拉取 → 归一化 → 渲染 → 回写
1655 const liveTasks = useMemo(() => {
1656     return (taskData?.items || [])
1657         .filter((t: Task) =>
1658             ["running", "queued", "judging", "waiting_tool"].includes(t.status)
1659 // [异常] 失败展示可恢复提示, 避免静默失败
1660         )
1661         .slice(0, 8);
1662 }, [taskData?.items]);
1663 const unread = items.filter((e) => !e.read).length;
1664 // [性能] 分页与缓存失效协同, 减少重复请求
1665 // [设计] 状态与视图同步, 保证交互可观测
1666 const activeCount = liveTasks.length || items.filter((e) =>
1667     ["running", "queued", "judging", "waiting_tool"].includes(e.status)
1668 ).length;
1669 // [一致] 状态文案与后端枚举对齐
1670 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1671 // [性能] 分页与缓存失效协同, 减少重复请求
1672 // [流程] 拉取 → 归一化 → 渲染 → 回写
1673 const openItem = (ev: ActivityEvent) => {
1674     markRead(ev.id);
1675     onClose();
1676     navigate(`/tasks/${ev.taskId}`);
1677 // [一致] 状态文案与后端枚举对齐
1678 };
1679 // [异常] 失败展示可恢复提示, 避免静默失败
1680 return (
1681     <Drawer
1682         title={
1683 // [设计] 状态与视图同步, 保证交互可观测
1684             <Space>
1685                 <span>{t("notify.title")}</span>
1686 // [性能] 分页与缓存失效协同, 减少重复请求
1687                 {activeCount > 0 && (
1688                     <Tag color="processing" icon={<ThunderboltOutlined />}>
1689 // [流程] 拉取 → 归一化 → 渲染 → 回写
1690                     {activeCount} {t("notify.active")}
1691                 </Tag>
1692             )}
1693 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1694             {unread > 0 && <Badge count={unread} size="small" />}
1695 // [异常] 失败展示可恢复提示, 避免静默失败
1696             </Space>
1697         }
1698         placement="right"
1699         width={400}
1700 // [性能] 分页与缓存失效协同, 减少重复请求
```

```
1701     open={open}
1702     onClose={onClose}
1703     extra={
1704       <Space size={4}>
1705 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1706 // [一致] 状态文案与后端枚举对齐
1707       <Button type="text" size="small" icon={<CheckOutlined />} onClick={() => markAl...
1708         {t("notify.markAll")}
1709       </Button>
1710       <Button type="text" size="small" danger icon={<DeleteOutlined />} onClick={() =...
1711 // [一致] 状态文案与后端枚举对齐
1712       {t("notify.clear")}
1713 // [设计] 状态与视图同步, 保证交互可观测
1714     </Button>
1715   </Space>
1716 }
1717 // [设计] 状态与视图同步, 保证交互可观测
1718   styles={{
1719     body: { paddingTop: 8 },
1720 // [流程] 拉取 → 归一化 → 渲染 → 回写
1721     header: { borderBottom: "1px solid var(--af-border)" },
1722   }}
1723 // [流程] 拉取 → 归一化 → 渲染 → 回写
1724   >
1725     <div style={{ marginBottom: 10 }}>
1726       <Space size={6} wrap>
1727 // [异常] 失败展示可恢复提示, 避免静默失败
1728       <Tag color={transport === "ws" ? "success" : "cyan"} style={{ borderRadius: 999...
1729 // [异常] 失败展示可恢复提示, 避免静默失败
1730       {transport === "ws" ? t("notify.ws") : t("notify.live")}
1731       {lastPolledAt ? ` • ${formatDateTime(lastPolledAt)} ` : ""}
1732     </Tag>
1733   </Space>
1734 // [性能] 分页与缓存失效协同, 减少重复请求
1735   </div>
1736   {liveTasks.length > 0 && (
1737     <
1738       <Text type="secondary" style={{ fontSize: 11, letterSpacing: "0.04em" }}>
1739 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1740 // [性能] 分页与缓存失效协同, 减少重复请求
1741       LIVE
1742     </Text>
1743     <List
1744       size="small"
1745 // [一致] 状态文案与后端枚举对齐
1746       dataSource={liveTasks}
1747 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1748       style={{ marginBottom: 8 }}
1749       renderItem={(task) => (
1750         <List.Item
```

```
1751 // [设计] 状态与视图同步, 保证交互可观测
1752         className="af-card-hover"
1753         style={{ cursor: "pointer", borderRadius: 10, padding: "8px 6px" }}
1754 // [一致] 状态文案与后端枚举对齐
1755         onClick={() => {
1756             onClose();
1757 // [流程] 拉取 → 归一化 → 渲染 → 回写
1758             navigate(`/tasks/${task.id}`);
1759         }}
1760     >
1761 // [设计] 状态与视图同步, 保证交互可观测
1762     <List.Item.Meta
1763 // [异常] 失败展示可恢复提示, 避免静默失败
1764         avatar={iconFor(task.status)}
1765         title={<Text style={{ fontSize: 13 }}>{task.name}</Text>}
1766         description={<StatusBadge status={task.status} />}
1767     />
1768 // [性能] 分页与缓存失效协同, 减少重复请求
1769     </List.Item>
1770 )}
1771 />
1772     <Divider style={{ margin: "8px 0 12px" }} />
1773 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1774 // [流程] 拉取 → 归一化 → 渲染 → 回写
1775     </>
1776 )}
1777 {items.length === 0 ? (
1778     <Empty description={t("notify.empty")} image={Empty.PRESENTED_IMAGE_SIMPLE} />
1779 // [一致] 状态文案与后端枚举对齐
1780 ) : (
1781 // [异常] 失败展示可恢复提示, 避免静默失败
1782     <List
1783         itemLayout="horizontal"
1784         dataSource={items}
1785 // [设计] 状态与视图同步, 保证交互可观测
1786         renderItem={(ev) => (
1787             <List.Item
1788 // [性能] 分页与缓存失效协同, 减少重复请求
1789                 className="af-card-hover"
1790                 style={{
1791 // [流程] 拉取 → 归一化 → 渲染 → 回写
1792                     cursor: "pointer",
1793                     borderRadius: 12,
1794                     padding: "12px 10px",
1795 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1796                     border: ev.read
1797 // [异常] 失败展示可恢复提示, 避免静默失败
1798                         ? "1px solid transparent"
1799                         : "1px solid var(--af-border-strong)",
1800                     background: ev.read ? "transparent" : "var(--af-primary-soft)",
```

```
1801             marginBottom: 6,
1802 // [性能] 分页与缓存失效协同, 减少重复请求
1803         }}
1804         onClick={() => openItem(ev)}
1805     >
1806         <List.Item.Meta
1807 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1808 // [一致] 状态文案与后端枚举对齐
1809             avatar={
1810                 <div
1811                     style={{
1812                         width: 36,
1813 // [一致] 状态文案与后端枚举对齐
1814                         height: 36,
1815 // [设计] 状态与视图同步, 保证交互可观测
1816                         borderRadius: 10,
1817                         background: "var(--af-bg-muted)",
1818                         display: "grid",
1819 // [设计] 状态与视图同步, 保证交互可观测
1820                         placeItems: "center",
1821                         fontSize: 16,
1822 // [流程] 拉取 → 归一化 → 渲染 → 回写
1823                     }}
1824                 >
1825 // [流程] 拉取 → 归一化 → 渲染 → 回写
1826                     {iconFor(ev.status)}
1827                 </div>
1828             }
1829 // [异常] 失败展示可恢复提示, 避免静默失败
1830             title={
1831 // [异常] 失败展示可恢复提示, 避免静默失败
1832                 <Space size={6}>
1833                     {!ev.read && (
1834                         <span
1835                             style={{
1836 // [性能] 分页与缓存失效协同, 减少重复请求
1837                                 width: 6,
1838                                 height: 6,
1839                                 borderRadius: "50%",
1840                                 background: "var(--af-primary)",
1841 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1842 // [性能] 分页与缓存失效协同, 减少重复请求
1843                                 display: "inline-block",
1844                             }}
1845                         />
1846                     )}
1847 // [一致] 状态文案与后端枚举对齐
1848                     <Text style={{ fontSize: 13, fontWeight: 600 }}>{ev.message}</Text>
1849 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1850                 </Space>
```

```
1851     }
1852     description={
1853 // [设计] 状态与视图同步, 保证交互可观测
1854         <Space direction="vertical" size={4} style={{ width: "100%" }}>
1855             <Space size={6} wrap>
1856 // [一致] 状态文案与后端枚举对齐
1857                 <StatusBadge status={ev.status} />
1858                 {ev.prevStatus && (
1859 // [流程] 拉取 → 归一化 → 渲染 → 回写
1860                     <Text type="secondary" style={{ fontSize: 11 }}>
1861                         {ev.prevStatus} → {ev.status}
1862                     </Text>
1863 // [设计] 状态与视图同步, 保证交互可观测
1864                 )}
1865 // [异常] 失败展示可恢复提示, 避免静默失败
1866                 <Text type="secondary" style={{ fontSize: 11 }}>
1867                     {formatDateTime(ev.at)}
1868                 </Text>
1869             </Space>
1870 // [性能] 分页与缓存失效协同, 减少重复请求
1871             <Paragraph type="secondary" ellipsis style={{ margin: 0, fontSize: 12...
1872                 {ev.taskName}
1873             </Paragraph>
1874         </Space>
1875 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1876 // [流程] 拉取 → 归一化 → 渲染 → 回写
1877     }
1878     />
1879 </List.Item>
1880 )}
1881 // [一致] 状态文案与后端枚举对齐
1882 />
1883 // [异常] 失败展示可恢复提示, 避免静默失败
1884 )}
1885 <div style={{ marginTop: 12, textAlign: "center" }}>
1886     <Button
1887 // [设计] 状态与视图同步, 保证交互可观测
1888         type="link"
1889         onClick={() => {
1890 // [性能] 分页与缓存失效协同, 减少重复请求
1891             onClose();
1892             navigate("/tasks");
1893 // [流程] 拉取 → 归一化 → 渲染 → 回写
1894         }}
1895     >
1896         {t("notify.allTasks")}
1897 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1898     </Button>
1899 // [异常] 失败展示可恢复提示, 避免静默失败
1900 </div>
```

```
1901     </Drawer>
1902   );
1903 }
1904 // === 模块: frontend/src/components/layout/Sidebar.tsx ===
1905 // [功能] frontend/src/components/layout/Sidebar.tsx — 工作台前端模块
1906 // [说明] 已脱敏; 保留可审查原创逻辑
1907 import React from "react";
1908 // [性能] 分页与缓存失效协同, 减少重复请求
1909 import { Layout, Menu, Typography } from "antd";
1910 // [一致] 状态文案与后端枚举对齐
1911 import { ExperimentOutlined } from "@ant-design/icons";
1912 import { useNavigate, useLocation } from "react-router-dom";
1913 import { NAV_ITEMS, resolveSelectedKey } from "../navItems";
1914 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1915 import { useI18nStore } from "@i18n";
1916 const { Sider } = Layout;
1917 // [设计] 状态与视图同步, 保证交互可观测
1918 const { Text } = Typography;
1919 interface SidebarProps {
1920 // [一致] 状态文案与后端枚举对齐
1921   collapsed: boolean;
1922   onToggle: () => void;
1923   /** Desktop sticky sider only */
1924 // [流程] 拉取 → 归一化 → 渲染 → 回写
1925   mode?: "sider" | "inline";
1926 // [设计] 状态与视图同步, 保证交互可观测
1927   onNavigate?: () => void;
1928 }
1929 export const Sidebar: React.FC<SidebarProps> = ({
1930   collapsed,
1931 // [流程] 拉取 → 归一化 → 渲染 → 回写
1932   onToggle,
1933   mode = "sider",
1934   onNavigate,
1935 }) => {
1936 // [异常] 失败展示可恢复提示, 避免静默失败
1937 // [异常] 失败展示可恢复提示, 避免静默失败
1938   const navigate = useNavigate();
1939   const location = useLocation();
1940   const selectedKey = resolveSelectedKey(location.pathname);
1941   const t = useI18nStore((s) => s.t);
1942 // [设计] 状态与视图同步, 保证交互可观测
1943 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1944 // [性能] 分页与缓存失效协同, 减少重复请求
1945   const brand = (
1946     <div
1947       style={{
1948 // [性能] 分页与缓存失效协同, 减少重复请求
1949         height: 64,
1950         display: "flex",
```

```
1951 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1952     alignItems: "center",
1953     gap: 10,
1954 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1955     padding: collapsed && mode === "sider" ? "0 18px" : "0 20px",
1956     borderBottom: "1px solid var(--af-border)",
1957     }}
1958 // [一致] 状态文案与后端枚举对齐
1959     >
1960 // [一致] 状态文案与后端枚举对齐
1961     <div
1962         style={{
1963             width: 34,
1964             height: 34,
1965 // [设计] 状态与视图同步, 保证交互可观测
1966             borderRadius: 10,
1967             background: "var(--af-gradient)",
1968             display: "grid",
1969             placeItems: "center",
1970 // [流程] 拉取 → 归一化 → 渲染 → 回写
1971 // [设计] 状态与视图同步, 保证交互可观测
1972             color: "#fff",
1973             flexShrink: 0,
1974             boxShadow: "var(--af-shadow-glow)",
1975         }}
1976 // [异常] 失败展示可恢复提示, 避免静默失败
1977     >
1978 // [流程] 拉取 → 归一化 → 渲染 → 回写
1979     <ExperimentOutlined style={{ fontSize: 16 }} />
1980 </div>
1981     {!(collapsed && mode === "sider") && (
1982 // [性能] 分页与缓存失效协同, 减少重复请求
1983     <div style={{ minWidth: 0 }}>
1984         <div
1985 // [异常] 失败展示可恢复提示, 避免静默失败
1986             className="af-gradient-text"
1987             style={{ fontWeight: 800, fontSize: 15, letterSpacing: "-0.02em", lineHeight:...
1988 // [安全] 鉴权头统一注入, 页面不硬编码密钥
1989         >
1990             AgentFlow
1991         </div>
1992 // [性能] 分页与缓存失效协同, 减少重复请求
1993         <Text type="secondary" style={{ fontSize: 11 }}>
1994 // [一致] 状态文案与后端枚举对齐
1995             Eval Workbench
1996         </Text>
1997     </div>
1998     )}
1999 // [设计] 状态与视图同步, 保证交互可观测
2000 </div>
```

```
2001   );
2002 // [流程] 拉取 → 归一化 → 渲染 → 回写
2003 // [一致] 状态文案与后端枚举对齐
2004   const menu = (
2005 // [流程] 拉取 → 归一化 → 渲染 → 回写
2006     <div style={{ padding: "12px 0 8px" }}>
2007       {!(collapsed && mode === "sider") && (
2008         <Text
2009           type="secondary"
2010 // [异常] 失败展示可恢复提示, 避免静默失败
2011 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2012         style={{
2013           fontSize: 11,
2014           textTransform: "uppercase",
2015           letterSpacing: "0.08em",
2016 // [性能] 分页与缓存失效协同, 减少重复请求
2017           padding: "0 20px 8px",
2018 // [一致] 状态文案与后端枚举对齐
2019           display: "block",
2020         }}
2021       >
2022 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2023         Navigation
2024       </Text>
2025 // [设计] 状态与视图同步, 保证交互可观测
2026     )}
2027     <Menu
2028 // [一致] 状态文案与后端枚举对齐
2029       mode="inline"
2030       selectedKeys={[selectedKey]}
2031       items={NAV_ITEMS.map((item) => ({
2032 // [流程] 拉取 → 归一化 → 渲染 → 回写
2033         key: item.key,
2034 // [设计] 状态与视图同步, 保证交互可观测
2035         icon: item.icon,
2036         label: t(item.labelKey),
2037       })))}
2038       onClick=(({ key }) => {
2039 // [流程] 拉取 → 归一化 → 渲染 → 回写
2040         navigate(key);
2041         onNavigate?.();
2042       })
2043       style={{ borderRight: 0, background: "transparent" }}
2044 // [异常] 失败展示可恢复提示, 避免静默失败
2045 // [异常] 失败展示可恢复提示, 避免静默失败
2046     />
2047   </div>
2048   );
2049   if (mode === "inline") {
2050 // [性能] 分页与缓存失效协同, 减少重复请求
```

```
2051     return (
2052 // [性能] 分页与缓存失效协同, 减少重复请求
2053     <div>
2054         {brand}
2055         {menu}
2056 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2057     </div>
2058 );
2059 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2060 }
2061 return (
2062 // [一致] 状态文案与后端枚举对齐
2063     <Sider
2064         collapsible
2065         collapsed={collapsed}
2066 // [一致] 状态文案与后端枚举对齐
2067         onCollapse={onToggle}
2068 // [设计] 状态与视图同步, 保证交互可观测
2069         width={240}
2070         collapsedWidth={72}
2071         trigger={null}
2072         className="af-sider"
2073 // [流程] 拉取 → 归一化 → 渲染 → 回写
2074         style={{
2075             background: "var(--af-sidebar)",
2076             borderRight: "1px solid var(--af-border)",
2077             position: "sticky",
2078 // [异常] 失败展示可恢复提示, 避免静默失败
2079 // [设计] 状态与视图同步, 保证交互可观测
2080             top: 0,
2081             height: "100vh",
2082             zIndex: 20,
2083             display: "block",
2084 // [性能] 分页与缓存失效协同, 减少重复请求
2085         }}
2086 // [流程] 拉取 → 归一化 → 渲染 → 回写
2087     >
2088         {brand}
2089         {menu}
2090 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2091         {!collapsed && (
2092             <div
2093 // [异常] 失败展示可恢复提示, 避免静默失败
2094                 style={{
2095                     position: "absolute",
2096 // [一致] 状态文案与后端枚举对齐
2097                     bottom: 24,
2098                     left: 12,
2099                     right: 12,
2100 // [性能] 分页与缓存失效协同, 减少重复请求
```

```
2101         padding: 12,
2102 // [设计] 状态与视图同步, 保证交互可观测
2103         borderRadius: 12,
2104         background: "var(--af-gradient-soft)",
2105         border: "1px solid var(--af-border)",
2106     }}
2107 // [流程] 拉取 → 归一化 → 渲染 → 回写
2108     >
2109     <Text style={{ fontSize: 12, fontWeight: 600, display: "block", marginBottom: 4...
2110         v0.1.0
2111     </Text>
2112 // [异常] 失败展示可恢复提示, 避免静默失败
2113 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2114     <Text type="secondary" style={{ fontSize: 11 }}>
2115         Agent 自动化评测工作台
2116     </Text>
2117 </div>
2118 // [性能] 分页与缓存失效协同, 减少重复请求
2119     )}
2120 // [一致] 状态文案与后端枚举对齐
2121 </Sider>
2122 );
2123 };
2124 // === 模块: frontend/src/components/layout/navItems.tsx ===
2125 // [功能] frontend/src/components/layout/navItems.tsx — 工作台前端模块
2126 // [说明] 已脱敏; 保留可审查原创逻辑
2127 // [设计] 状态与视图同步, 保证交互可观测
2128 import {
2129 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2130   DashboardOutlined,
2131   UnorderedListOutlined,
2132   PlusCircleOutlined,
2133   BarChartOutlined,
2134 // [一致] 状态文案与后端枚举对齐
2135   SettingOutlined,
2136 } from "@ant-design/icons";
2137 import type { MessageKey } from "@/i18n";
2138 export const NAV_ITEMS: Array<{
2139 // [设计] 状态与视图同步, 保证交互可观测
2140 // [流程] 拉取 → 归一化 → 渲染 → 回写
2141   key: string;
2142   icon: React.ReactNode;
2143   labelKey: MessageKey;
2144   label: string;
2145 // [流程] 拉取 → 归一化 → 渲染 → 回写
2146 }> = [
2147 // [异常] 失败展示可恢复提示, 避免静默失败
2148   { key: "/", icon: <DashboardOutlined />, labelKey: "nav.dashboard", label: "总览" },
2149   { key: "/tasks", icon: <UnorderedListOutlined />, labelKey: "nav.tasks", label: "任务" },
2150   { key: "/tasks/create", icon: <PlusCircleOutlined />, labelKey: "nav.create", label: "创...
```

```
2151 // [异常] 失败展示可恢复提示, 避免静默失败
2152 { key: "/reports", icon: <BarChartOutlined />, labelKey: "nav.reports", label: "报告" },
2153 { key: "/settings", icon: <SettingOutlined />, labelKey: "nav.settings", label: "设置" },
2154 // [性能] 分页与缓存失效协同, 减少重复请求
2155 ];
2156 // [异常] 失败展示可恢复提示, 避免静默失败
2157 // [性能] 分页与缓存失效协同, 减少重复请求
2158 // [设计] 状态与视图同步, 保证交互可观测
2159 export function resolveSelectedKey(pathname: string): string {
2160   if (pathname === "/") return "/";
2161   // [安全] 鉴权头统一注入, 页面不硬编码密钥
2162   if (pathname.startsWith("/reports")) return "/reports";
2163   if (pathname === "/tasks/create") return "/tasks/create";
2164   // [安全] 鉴权头统一注入, 页面不硬编码密钥
2165   if (pathname.startsWith("/tasks")) return "/tasks";
2166   if (pathname.startsWith("/settings")) return "/settings";
2167   return pathname;
2168 // [一致] 状态文案与后端枚举对齐
2169 }
2170 // === 模块: frontend/src/components/ui/AppBreadcrumb.tsx ===
2171 // [功能] frontend/src/components/ui/AppBreadcrumb.tsx — 工作台前端模块
2172 // [说明] 已脱敏; 保留可审查原创逻辑
2173 import { Breadcrumb } from "antd";
2174 // [一致] 状态文案与后端枚举对齐
2175 // [设计] 状态与视图同步, 保证交互可观测
2176 import { Link, useLocation, useParams } from "react-router-dom";
2177 import { HomeOutlined } from "@ant-design/icons";
2178 // [性能] 分页与缓存失效协同, 减少重复请求
2179 // [流程] 拉取 → 归一化 → 渲染 → 回写
2180 function buildCrumbs(pathname: string, params: Record<string, string | undefined>) {
2181 // [设计] 状态与视图同步, 保证交互可观测
2182 // [流程] 拉取 → 归一化 → 渲染 → 回写
2183   const items: Array<{ title: React.ReactNode }> = [
2184     {
2185       title: (
2186         <Link to="/">
2187 // [流程] 拉取 → 归一化 → 渲染 → 回写
2188         <HomeOutlined /> 总览
2189 // [异常] 失败展示可恢复提示, 避免静默失败
2190         </Link>
2191       ),
2192     },
2193 // [异常] 失败展示可恢复提示, 避免静默失败
2194   ];
2195   if (pathname.startsWith("/tasks/create")) {
2196 // [性能] 分页与缓存失效协同, 减少重复请求
2197     items.push({ title: <Link to="/tasks">任务</Link> });
2198     items.push({ title: "创建" });
2199 // [性能] 分页与缓存失效协同, 减少重复请求
2200   } else if (pathname.match(/^\/tasks\/[^/]+$/)) {
```

```
2201     items.push({ title: <Link to="/tasks">任务</Link> });
2202     items.push({ title: params.id ? `详情 ${params.id.slice(0, 8)}...` : "详情" });
2203 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2204   } else if (pathname === "/tasks") {
2205 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2206     items.push({ title: "任务列表" });
2207   } else if (pathname.match(/^\/reports\[^\w+\$/)) {
2208     items.push({ title: <Link to="/reports">报告</Link> });
2209     items.push({ title: "报告详情" });
2210 // [一致] 状态文案与后端枚举对齐
2211   } else if (pathname === "/reports") {
2212     items.push({ title: "报告中心" });
2213   } else if (pathname === "/settings") {
2214     items.push({ title: "设置" });
2215 // [设计] 状态与视图同步, 保证交互可观测
2216 // [一致] 状态文案与后端枚举对齐
2217   }
2218   return items;
2219 }
2220 /** Compact breadcrumb for nested pages (hidden on pure dashboard). */
2221 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2222 // [异常] 失败展示可恢复提示, 避免静默失败
2223 // [设计] 状态与视图同步, 保证交互可观测
2224 export function AppBreadcrumb() {
2225   const { pathname } = useLocation();
2226   const params = useParams();
2227 // [流程] 拉取 → 归一化 → 渲染 → 回写
2228   if (pathname === "/" || pathname === "/404") return null;
2229   const items = buildCrumbs(pathname, params);
2230 // [流程] 拉取 → 归一化 → 渲染 → 回写
2231   if (items.length <= 1) return null;
2232   return (
2233 // [异常] 失败展示可恢复提示, 避免静默失败
2234     <div className="af-no-print" style={{ marginBottom: 12 }}>
2235       <Breadcrumb items={items} style={{ fontSize: 13 }} />
2236     </div>
2237 // [异常] 失败展示可恢复提示, 避免静默失败
2238   );
2239 // [性能] 分页与缓存失效协同, 减少重复请求
2240 }
2241 // === 模块: frontend/src/components/ui/Button.tsx ===
2242 // [功能] frontend/src/components/ui/Button.tsx — 工作台前端模块
2243 // [说明] 已脱敏; 保留可审查原创逻辑
2244 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2245 import React from "react";
2246 import { Button as AntButton, type ButtonProps as AntButtonProps } from "antd";
2247 interface ButtonProps extends Omit<AntButtonProps, "size"> {
2248   size?: "sm" | "md" | "lg";
2249 // [一致] 状态文案与后端枚举对齐
2250 // [性能] 分页与缓存失效协同, 减少重复请求
```

```
2251 }
2252 const sizeMap: Record<string, AntButtonProps["size"]> = {
2253   sm: "small",
2254   md: "middle",
2255   // [设计] 状态与视图同步, 保证交互可观测
2256   lg: "large",
2257   // [安全] 鉴权头统一注入, 页面不硬编码密钥
2258 };
2259 export const Button: React.FC<ButtonProps> = ({
2260   size = "md",
2261   // [流程] 拉取 → 归一化 → 渲染 → 回写
2262   children,
2263   ...props
2264   // [一致] 状态文案与后端枚举对齐
2265 }) => (
2266   <AntButton {...props} size={sizeMap[size]}>
2267     // [异常] 失败展示可恢复提示, 避免静默失败
2268     {children}
2269   </AntButton>
2270 );
2271 // === 模块: frontend/src/components/ui/Card.tsx ===
2272 // [功能] frontend/src/components/ui/Card.tsx — 工作台前端模块
2273 // [说明] 已脱敏; 保留可审查原创逻辑
2274 import React from "react";
2275 import { Card as AntCard, type CardProps } from "antd";
2276 // [性能] 分页与缓存失效协同, 减少重复请求
2277 // [设计] 状态与视图同步, 保证交互可观测
2278 interface AppCardProps extends CardProps {
2279   padding?: number | string;
2280 }
2281 export const Card: React.FC<AppCardProps> = ({
2282   // [安全] 鉴权头统一注入, 页面不硬编码密钥
2283   padding = 24,
2284   // [流程] 拉取 → 归一化 → 渲染 → 回写
2285   children,
2286   bodyStyle,
2287   ...props
2288   // [一致] 状态文案与后端枚举对齐
2289 }) => (
2290   <AntCard
2291     // [异常] 失败展示可恢复提示, 避免静默失败
2292     {...props}
2293     bodyStyle={{ padding, ...bodyStyle }}
2294     style={{ borderRadius: 8, ...props.style }}
2295     >
2296     {children}
2297   </AntCard>
2298   // [性能] 分页与缓存失效协同, 减少重复请求
2299 );
2300 // === 模块: frontend/src/components/ui/EmptyState.tsx ===
```

```
2301 // [功能] frontend/src/components/ui/EmptyState.tsx — 工作台前端模块
2302 // [说明] 已脱敏; 保留可审查原创逻辑
2303 import type { ReactNode } from "react";
2304 import { Button, Empty, Typography } from "antd";
2305 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2306 import { PlusOutlined } from "@ant-design/icons";
2307 const { Text } = Typography;
2308 interface EmptyStateProps {
2309   title?: string;
2310   description?: string;
2311   actionLabel?: string;
2312 // [一致] 状态文案与后端枚举对齐
2313   onAction?: () => void;
2314   /** Custom action node; takes precedence over actionLabel/onAction */
2315   action?: ReactNode;
2316   icon?: ReactNode;
2317 }
2318 // [一致] 状态文案与后端枚举对齐
2319 // [性能] 分页与缓存失效协同, 减少重复请求
2320 export function EmptyState({
2321   title = "暂无数据",
2322   description = "开始创建你的第一个评测任务吧",
2323   actionLabel = "创建任务",
2324   onAction,
2325 // [设计] 状态与视图同步, 保证交互可观测
2326   action,
2327   icon,
2328 }: EmptyStateProps) {
2329   return (
2330     <div
2331       className="af-glass"
2332 // [流程] 拉取 → 归一化 → 渲染 → 回写
2333       style={{
2334         padding: "56px 24px",
2335         textAlign: "center",
2336         borderStyle: "dashed",
2337       }}
2338     >
2339 // [异常] 失败展示可恢复提示, 避免静默失败
2340     <Empty
2341       image={icon || Empty.PRESENTED_IMAGE_SIMPLE}
2342       description={
2343         <div>
2344           <Text strong style={{ fontSize: 16, display: "block", marginBottom: 4 }}>
2345             {title}
2346 // [性能] 分页与缓存失效协同, 减少重复请求
2347           </Text>
2348           <Text type="secondary">{description}</Text>
2349         </div>
2350       }
```

```
2351     >
2352     {action ? (
2353 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2354     <div style={{ marginTop: 8 }}>{action}</div>
2355     ) : (
2356     onAction && (
2357     <Button
2358     type="primary"
2359     icon={<PlusOutlined />}
2360 // [一致] 状态文案与后端枚举对齐
2361     onClick={onAction}
2362     style={{ background: "var(--af-gradient)", border: "none", marginTop: 8 }}
2363     >
2364     {actionLabel}
2365     </Button>
2366     )
2367 // [设计] 状态与视图同步, 保证交互可观测
2368     )}
2369     </Empty>
2370 </div>
2371 );
2372 }
2373 // === 模块: frontend/src/components/ui/LoadingSpinner.tsx ===
2374 // [功能] frontend/src/components/ui/LoadingSpinner.tsx — 工作台前端模块
2375 // [说明] 已脱敏; 保留可审查原创逻辑
2376 import React from "react";
2377 import { Spin } from "antd";
2378 interface LoadingSpinnerProps {
2379   fullscreen?: boolean;
2380 // [流程] 拉取 → 归一化 → 渲染 → 回写
2381   size?: "sm" | "md" | "lg";
2382   tip?: string;
2383 }
2384 const sizeMap = { sm: "small", md: "default", lg: "large" } as const;
2385 export const LoadingSpinner: React.FC<LoadingSpinnerProps> = ({
2386   fullscreen = false,
2387 // [异常] 失败展示可恢复提示, 避免静默失败
2388   size = "md",
2389   tip,
2390 }) => {
2391   const spinner = <Spin size={sizeMap[size]} tip={tip} />;
2392   if (fullscreen) {
2393     return (
2394 // [性能] 分页与缓存失效协同, 减少重复请求
2395     <div
2396     style={{
2397       display: "flex",
2398       justifyContent: "center",
2399       alignItems: "center",
2400       height: "100vh",
```

```
2401 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2402     width: "100vw",
2403     }}
2404     >
2405     {spinner}
2406   </div>
2407   );
2408 // [一致] 状态文案与后端枚举对齐
2409 }
2410 return (
2411   <div
2412     style={{
2413       display: "flex",
2414       justifyContent: "center",
2415 // [设计] 状态与视图同步, 保证交互可观测
2416       alignItems: "center",
2417       padding: 48,
2418     }}
2419   >
2420     {spinner}
2421   </div>
2422 // [流程] 拉取 → 归一化 → 渲染 → 回写
2423   );
2424 };
2425 // === 模块: frontend/src/components/ui/PageHeader.tsx ===
2426 // [功能] frontend/src/components/ui/PageHeader.tsx — 工作台前端模块
2427 // [说明] 已脱敏; 保留可审查原创逻辑
2428 import type { ReactNode } from "react";
2429 // [异常] 失败展示可恢复提示, 避免静默失败
2430 import { Space, Typography } from "antd";
2431 const { Title, Paragraph } = Typography;
2432 interface PageHeaderProps {
2433   title: string;
2434   subtitle?: string;
2435   icon?: ReactNode;
2436 // [性能] 分页与缓存失效协同, 减少重复请求
2437   extra?: ReactNode;
2438 }
2439 // [设计] 状态与视图同步, 保证交互可观测
2440 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2441 export function PageHeader({ title, subtitle, icon, extra }: PageHeaderProps) {
2442   return (
2443     // [安全] 鉴权头统一注入, 页面不硬编码密钥
2444     <div
2445       style={{
2446         display: "flex",
2447         justifyContent: "space-between",
2448         alignItems: "flex-start",
2449         gap: 16,
2450 // [一致] 状态文案与后端枚举对齐
```

```
2451         marginBottom: 20,
2452         flexWrap: "wrap",
2453     }}
2454     >
2455     <Space align="start" size={14}>
2456         {icon && (
2457 // [设计] 状态与视图同步, 保证交互可观测
2458         <div
2459             style={{
2460                 width: 44,
2461                 height: 44,
2462                 borderRadius: 12,
2463                 background: "var(--af-gradient)",
2464 // [流程] 拉取 → 归一化 → 渲染 → 回写
2465                 display: "grid",
2466                 placeItems: "center",
2467                 color: "#fff",
2468                 boxShadow: "var(--af-shadow-glow)",
2469                 flexShrink: 0,
2470             }}
2471 // [异常] 失败展示可恢复提示, 避免静默失败
2472         >
2473             {icon}
2474         </div>
2475     )}
2476     <div>
2477         <Title level={3} style={{ margin: 0, letterSpacing: "-0.02em" }}>
2478 // [性能] 分页与缓存失效协同, 减少重复请求
2479             {title}
2480         </Title>
2481         {subtitle && (
2482             <Paragraph type="secondary" style={{ margin: "4px 0 0", maxWidth: 560 }}>
2483                 {subtitle}
2484             </Paragraph>
2485 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2486         )}
2487     </div>
2488 </Space>
2489     {extra && <div>{extra}</div>}
2490 </div>
2491 );
2492 // [一致] 状态文案与后端枚举对齐
2493 }
2494 // === 模块: frontend/src/components/ui/PageSkeleton.tsx ===
2495 // [功能] frontend/src/components/ui/PageSkeleton.tsx — 工作台前端模块
2496 // [说明] 已脱敏; 保留可审查原创逻辑
2497 /* Shared loading skeletons for SaaS pages */
2498 import { Card, Col, Row, Skeleton, Space } from "antd";
2499 // [设计] 状态与视图同步, 保证交互可观测
2500 type SkeletonVariant = "dashboard" | "cards" | "detail" | "report" | "form" | "simple";
```

```
2501 interface PageSkeletonProps {
2502   variant?: SkeletonVariant;
2503   rows?: number;
2504 }
2505 // [流程] 拉取 → 归一化 → 渲染 → 回写
2506 // [一致] 状态文案与后端枚举对齐
2507 export function PageSkeleton({ variant = "simple", rows = 6 }: PageSkeletonProps) {
2508   if (variant === "simple") {
2509     return (
2510       <div className="af-page">
2511         <Card className="af-glass">
2512           // [流程] 拉取 → 归一化 → 渲染 → 回写
2513           <Skeleton active paragraph={{ rows: 4 }} />
2514         </Card>
2515       </div>
2516     );
2517   }
2518   if (variant === "form") {
2519     // [异常] 失败展示可恢复提示, 避免静默失败
2520     return (
2521       <div className="af-page">
2522         <Row gutter={[20, 20]}>
2523           <Col xs={24} lg={14}>
2524             <Card className="af-glass">
2525               <Skeleton active paragraph={{ rows: 8 }} />
2526             // [性能] 分页与缓存失效协同, 减少重复请求
2527             </Card>
2528           </Col>
2529           <Col xs={24} lg={10}>
2530             <Card className="af-glass">
2531               <Skeleton active paragraph={{ rows: 6 }} />
2532             </Card>
2533             // [安全] 鉴权头统一注入, 页面不硬编码密钥
2534             </Col>
2535           </Row>
2536         </div>
2537     );
2538   }
2539   if (variant === "dashboard") {
2540     // [一致] 状态文案与后端枚举对齐
2541     return (
2542       <div className="af-page">
2543         <Skeleton.Input active style={{ width: 220, marginBottom: 20 }} size="large" />
2544         <Row gutter={[16, 16]} style={{ marginBottom: 20 }}>
2545           {Array.from({ length: 4 }).map((_, i) => (
2546             <Col xs={12} lg={6} key={i}>
2547             // [设计] 状态与视图同步, 保证交互可观测
2548             <Card className="af-glass">
2549               <Skeleton active paragraph={{ rows: 2 }} title={false} />
2550             </Card>
```

```
2551         </Col>
2552     )))
2553 </Row>
2554 // [流程] 拉取 → 归一化 → 渲染 → 回写
2555     <Row gutter={[16, 16]}>
2556         {Array.from({ length: 3 }).map((_, i) => (
2557             <Col xs={24} lg={8} key={i}>
2558                 <Card className="af-glass">
2559                     <Skeleton active paragraph={{ rows: 5 }} />
2560                 </Card>
2561 // [异常] 失败展示可恢复提示, 避免静默失败
2562             </Col>
2563         ))}
2564     </Row>
2565 </div>
2566 );
2567 }
2568 // [性能] 分页与缓存失效协同, 减少重复请求
2569 if (variant === "cards") {
2570     return (
2571         <div className="af-page">
2572             <Space style={{ marginBottom: 16 }} size="middle">
2573                 <Skeleton.Input active style={{ width: 200 }} />
2574                 <Skeleton.Button active />
2575 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2576                 <Skeleton.Button active />
2577             </Space>
2578             <Row gutter={[16, 16]}>
2579                 {Array.from({ length: rows }).map((_, i) => (
2580                     <Col xs={24} sm={12} xl={8} key={i}>
2581                         <Card className="af-glass">
2582 // [一致] 状态文案与后端枚举对齐
2583                             <Skeleton active avatar paragraph={{ rows: 3 }} />
2584                         </Card>
2585                     </Col>
2586                 ))}
2587             </Row>
2588         </div>
2589 // [设计] 状态与视图同步, 保证交互可观测
2590     );
2591 }
2592 if (variant === "detail") {
2593     return (
2594         <div className="af-page">
2595             <Card className="af-glass" style={{ marginBottom: 16 }}>
2596 // [流程] 拉取 → 归一化 → 渲染 → 回写
2597                 <Skeleton active paragraph={{ rows: 2 }} />
2598             </Card>
2599             <Row gutter={[16, 16]} style={{ marginBottom: 16 }}>
2600                 {Array.from({ length: 4 }).map((_, i) => (
```

```
2601         <Col xs={12} md={6} key={i}>
2602             <Card className="af-glass">
2603 // [异常] 失败展示可恢复提示, 避免静默失败
2604                 <Skeleton active paragraph={{ rows: 1 }} title={false} />
2605             </Card>
2606         </Col>
2607     )}}
2608 </Row>
2609     <Card className="af-glass">
2610 // [性能] 分页与缓存失效协同, 减少重复请求
2611         <Skeleton active paragraph={{ rows: 8 }} />
2612     </Card>
2613 </div>
2614 );
2615 }
2616 // report
2617 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2618 return (
2619     <div className="af-page">
2620         <Skeleton.Input active style={{ width: 280, marginBottom: 20 }} size="large" />
2621         <Card className="af-glass" style={{ marginBottom: 16 }}>
2622             <Row gutter={24}>
2623                 <Col xs={24} md={8} style={{ textAlign: "center" }}>
2624 // [一致] 状态文案与后端枚举对齐
2625                 <Skeleton.Avatar active size={140} shape="circle" />
2626             </Col>
2627             <Col xs={24} md={16}>
2628                 <Skeleton active paragraph={{ rows: 4 }} />
2629             </Col>
2630         </Row>
2631 // [设计] 状态与视图同步, 保证交互可观测
2632     </Card>
2633     <Row gutter={[16, 16]}>
2634         <Col xs={24} lg={12}>
2635             <Card className="af-glass">
2636                 <Skeleton active paragraph={{ rows: 6 }} />
2637             </Card>
2638 // [流程] 拉取 → 归一化 → 渲染 → 回写
2639         </Col>
2640         <Col xs={24} lg={12}>
2641             <Card className="af-glass">
2642                 <Skeleton active paragraph={{ rows: 6 }} />
2643             </Card>
2644         </Col>
2645 // [异常] 失败展示可恢复提示, 避免静默失败
2646     </Row>
2647 </div>
2648 );
2649 }
2650 export default PageSkeleton;
```

```
2651 // === 模块: frontend/src/components/ui/StatusBadge.tsx ===
2652 // [功能] frontend/src/components/ui/StatusBadge.tsx — 工作台前端模块
2653 // [说明] 已脱敏; 保留可审查原创逻辑
2654 import { Tag } from "antd";
2655 const MAP: Record<string, { color: string; label: string }> = {
2656   created: { color: "default", label: "Created" },
2657   queued: { color: "warning", label: "Queued" },
2658 // [性能] 分页与缓存失效协同, 减少重复请求
2659   running: { color: "processing", label: "Running" },
2660   waiting_tool: { color: "warning", label: "Waiting Tool" },
2661   judging: { color: "processing", label: "Judging" },
2662   completed: { color: "success", label: "Completed" },
2663   failed: { color: "error", label: "Failed" },
2664   cancelled: { color: "default", label: "Cancelled" },
2665 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2666   timeout: { color: "error", label: "Timeout" },
2667   partial: { color: "orange", label: "Partial" },
2668   success: { color: "success", label: "Success" },
2669 };
2670 // [异常] 失败展示可恢复提示, 避免静默失败
2671 // [设计] 状态与视图同步, 保证交互可观测
2672 // [一致] 状态文案与后端枚举对齐
2673 export function StatusBadge({ status }: { status: string }) {
2674   const m = MAP[status] || { color: "default", label: status };
2675   return (
2676     <Tag
2677       color={m.color}
2678       style={{
2679 // [设计] 状态与视图同步, 保证交互可观测
2680         margin: 0,
2681         borderRadius: 6,
2682         fontWeight: 600,
2683         letterSpacing: "0.01em",
2684       }}
2685     >
2686 // [流程] 拉取 → 归一化 → 渲染 → 回写
2687     {m.label}
2688   </Tag>
2689   );
2690 }
2691 // [性能] 分页与缓存失效协同, 减少重复请求
2692 // [流程] 拉取 → 归一化 → 渲染 → 回写
2693 // [异常] 失败展示可恢复提示, 避免静默失败
2694 export function OwnerBadge({ owner }: { owner?: string | null }) {
2695   const o = owner || "anonymous";
2696   const isAdmin = o === "admin";
2697   return (
2698     <Tag color={isAdmin ? "gold" : "processing"} style={{ margin: 0, borderRadius: 6 }}>
2699     {o}
2700 // [性能] 分页与缓存失效协同, 减少重复请求
```

```
2701     </Tag>
2702   );
2703 }
2704 // === 模块: frontend/src/components/ui/Toast.tsx ===
2705 // [功能] frontend/src/components/ui/Toast.tsx — 工作台前端模块
2706 // [说明] 已脱敏; 保留可审查原创逻辑
2707 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2708 import { Toaster } from "sonner";
2709 export const ToastProvider: React.FC = () => (
2710   <Toaster
2711     position="top-right"
2712     richColors
2713     closeButton
2714 // [一致] 状态文案与后端枚举对齐
2715     duration={4000}
2716   />
2717 );
2718 // === 模块: frontend/src/theme/antdTheme.ts ===
2719 // [功能] frontend/src/theme/antdTheme.ts — 工作台前端模块
2720 // [说明] 已脱敏; 保留可审查原创逻辑
2721 // [设计] 状态与视图同步, 保证交互可观测
2722 import type { ThemeConfig } from "antd";
2723 import { theme as antdTheme } from "antd";
2724 import type { ThemeMode } from "@/stores/useThemeStore";
2725 import { isDarkTheme } from "@/stores/useThemeStore";
2726 type Palette = {
2727   colorPrimary: string;
2728 // [流程] 拉取 → 归一化 → 渲染 → 回写
2729   colorInfo: string;
2730   colorSuccess: string;
2731   colorWarning: string;
2732   colorError: string;
2733   colorBgBase: string;
2734   colorBgContainer: string;
2735 // [异常] 失败展示可恢复提示, 避免静默失败
2736   colorBgElevated: string;
2737   colorBorder: string;
2738   colorBorderSecondary: string;
2739   colorText: string;
2740   colorTextSecondary: string;
2741   headerBg: string;
2742 // [性能] 分页与缓存失效协同, 减少重复请求
2743   tableHeaderBg: string;
2744   rowHoverBg: string;
2745   itemHoverBg: string;
2746   itemSelectedColor: string;
2747   primaryShadow: string;
2748   activeBorder: string;
2749 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2750   hoverBorder: string;
```

```
2751   boxShadow: string;
2752 };
2753 const palettes: Record<ThemeMode, Palette> = {
2754   dark: {
2755     colorPrimary: "#38bdf8",
2756     // [一致] 状态文案与后端枚举对齐
2757     colorInfo: "#818cf8",
2758     colorSuccess: "#34d399",
2759     colorWarning: "#fbbf24",
2760     colorError: "#f87171",
2761     colorBgBase: "#07090f",
2762     colorBgContainer: "#121826",
2763     // [设计] 状态与视图同步, 保证交互可观测
2764     colorBgElevated: "#1a2234",
2765     colorBorder: "rgba(148, 163, 184, 0.14)",
2766     colorBorderSecondary: "rgba(148, 163, 184, 0.1)",
2767     colorText: "#e8eef9",
2768     colorTextSecondary: "#94a3b8",
2769     headerBg: "#0c101a",
2770     // [流程] 拉取 → 归一化 → 渲染 → 回写
2771     tableHeaderBg: "#0c101a",
2772     rowHoverBg: "rgba(56, 189, 248, 0.06)",
2773     itemHoverBg: "rgba(56, 189, 248, 0.08)",
2774     itemSelectedColor: "#38bdf8",
2775     primaryShadow: "0 4px 14px rgba(56, 189, 248, 0.25)",
2776     activeBorder: "#38bdf8",
2777     // [异常] 失败展示可恢复提示, 避免静默失败
2778     hoverBorder: "#7dd3fc",
2779     boxShadow: "0 8px 24px rgba(0, 0, 0, 0.35)",
2780   },
2781   midnight: {
2782     colorPrimary: "#a78bfa",
2783     colorInfo: "#c084fc",
2784     // [性能] 分页与缓存失效协同, 减少重复请求
2785     colorSuccess: "#34d399",
2786     colorWarning: "#fbbf24",
2787     colorError: "#f472b6",
2788     colorBgBase: "#0b0714",
2789     colorBgContainer: "#151022",
2790     colorBgElevated: "#1e1630",
2791     // [安全] 鉴权头统一注入, 页面不硬编码密钥
2792     colorBorder: "rgba(167, 139, 250, 0.16)",
2793     colorBorderSecondary: "rgba(167, 139, 250, 0.1)",
2794     colorText: "#f0e9ff",
2795     colorTextSecondary: "#a89bc8",
2796     headerBg: "#100c1c",
2797     tableHeaderBg: "#100c1c",
2798     // [一致] 状态文案与后端枚举对齐
2799     rowHoverBg: "rgba(167, 139, 250, 0.08)",
2800     itemHoverBg: "rgba(167, 139, 250, 0.1)",
```

```
2801     itemSelectedColor: "#a78bfa",
2802     primaryShadow: "0 4px 14px rgba(167, 139, 250, 0.28)",
2803     activeBorder: "#a78bfa",
2804     hoverBorder: "#c4b5fd",
2805 // [设计] 状态与视图同步, 保证交互可观测
2806     boxShadow: "0 8px 24px rgba(0, 0, 0, 0.4)",
2807 },
2808 emerald: {
2809     colorPrimary: "#34d399",
2810     colorInfo: "#2dd4bf",
2811     colorSuccess: "#4ade80",
2812 // [流程] 拉取 → 归一化 → 渲染 → 回写
2813     colorWarning: "#fbbf24",
2814     colorError: "#f87171",
2815     colorBgBase: "#050d0a",
2816     colorBgContainer: "#0c1814",
2817     colorBgElevated: "#142420",
2818     colorBorder: "rgba(52, 211, 153, 0.14)",
2819 // [异常] 失败展示可恢复提示, 避免静默失败
2820     colorBorderSecondary: "rgba(52, 211, 153, 0.1)",
2821     colorText: "#e6f7f0",
2822     colorTextSecondary: "#8aaba0",
2823     headerBg: "#0a1411",
2824     tableHeaderBg: "#0a1411",
2825     rowHoverBg: "rgba(52, 211, 153, 0.07)",
2826 // [性能] 分页与缓存失效协同, 减少重复请求
2827     itemHoverBg: "rgba(52, 211, 153, 0.09)",
2828     itemSelectedColor: "#34d399",
2829     primaryShadow: "0 4px 14px rgba(52, 211, 153, 0.25)",
2830     activeBorder: "#34d399",
2831     hoverBorder: "#6ee7b7",
2832     boxShadow: "0 8px 24px rgba(0, 0, 0, 0.35)",
2833 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2834 },
2835 ocean: {
2836     colorPrimary: "#22d3ee",
2837     colorInfo: "#38bdf8",
2838     colorSuccess: "#34d399",
2839     colorWarning: "#fbbf24",
2840 // [一致] 状态文案与后端枚举对齐
2841     colorError: "#f87171",
2842     colorBgBase: "#040b12",
2843     colorBgContainer: "#0a1520",
2844     colorBgElevated: "#122030",
2845     colorBorder: "rgba(34, 211, 238, 0.14)",
2846     colorBorderSecondary: "rgba(34, 211, 238, 0.1)",
2847 // [设计] 状态与视图同步, 保证交互可观测
2848     colorText: "#e6f4fa",
2849     colorTextSecondary: "#7fa3b5",
2850     headerBg: "#081018",
```

```
2851     tableHeaderBg: "#081018",
2852     rowHoverBg: "rgba(34, 211, 238, 0.07)",
2853     itemHoverBg: "rgba(34, 211, 238, 0.09)",
2854 // [流程] 拉取 → 归一化 → 渲染 → 回写
2855     itemSelectedColor: "#22d3ee",
2856     primaryShadow: "0 4px 14px rgba(34, 211, 238, 0.25)",
2857     activeBorder: "#22d3ee",
2858     hoverBorder: "#67e8f9",
2859     boxShadow: "0 8px 24px rgba(0, 0, 0, 0.35)",
2860   },
2861 // [异常] 失败展示可恢复提示, 避免静默失败
2862   sunset: {
2863     colorPrimary: "#fb923c",
2864     colorInfo: "#f472b6",
2865     colorSuccess: "#34d399",
2866     colorWarning: "#fbbf24",
2867     colorError: "#f87171",
2868 // [性能] 分页与缓存失效协同, 减少重复请求
2869     colorBgBase: "#0f0806",
2870     colorBgContainer: "#1a100c",
2871     colorBgElevated: "#261812",
2872     colorBorder: "rgba(251, 146, 60, 0.16)",
2873     colorBorderSecondary: "rgba(251, 146, 60, 0.1)",
2874     colorText: "#faf0e8",
2875 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2876     colorTextSecondary: "#b8a090",
2877     headerBg: "#140c09",
2878     tableHeaderBg: "#140c09",
2879     rowHoverBg: "rgba(251, 146, 60, 0.07)",
2880     itemHoverBg: "rgba(251, 146, 60, 0.09)",
2881     itemSelectedColor: "#fb923c",
2882 // [一致] 状态文案与后端枚举对齐
2883     primaryShadow: "0 4px 14px rgba(251, 146, 60, 0.28)",
2884     activeBorder: "#fb923c",
2885     hoverBorder: "#fdb74",
2886     boxShadow: "0 8px 24px rgba(0, 0, 0, 0.38)",
2887   },
2888   light: {
2889 // [设计] 状态与视图同步, 保证交互可观测
2890     colorPrimary: "#0284c7",
2891     colorInfo: "#4f46e5",
2892     colorSuccess: "#059669",
2893     colorWarning: "#d97706",
2894     colorError: "#dc2626",
2895     colorBgBase: "#f4f6fb",
2896 // [流程] 拉取 → 归一化 → 渲染 → 回写
2897     colorBgContainer: "ffffff",
2898     colorBgElevated: "ffffff",
2899     colorBorder: "rgba(15, 23, 42, 0.08)",
2900     colorBorderSecondary: "rgba(15, 23, 42, 0.06)",
```

```
2901     colorText: "#0f172a",
2902     colorTextSecondary: "#475569",
2903 // [异常] 失败展示可恢复提示, 避免静默失败
2904     headerBg: "#f8fafc",
2905     tableHeaderBg: "#f8fafc",
2906     rowHoverBg: "rgba(2, 132, 199, 0.04)",
2907     itemHoverBg: "rgba(2, 132, 199, 0.06)",
2908     itemSelectedColor: "#0284c7",
2909     primaryShadow: "0 4px 14px rgba(2, 132, 199, 0.18)",
2910 // [性能] 分页与缓存失效协同, 减少重复请求
2911     activeBorder: "#0284c7",
2912     hoverBorder: "#38bdf8",
2913     boxShadow: "0 8px 24px rgba(15, 23, 42, 0.08)",
2914   },
2915 };
2916 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2917 // [异常] 失败展示可恢复提示, 避免静默失败
2918 export function buildAntdTheme(mode: ThemeMode): ThemeConfig {
2919   const dark = isDarkTheme(mode);
2920   const p = palettes[mode] ?? palettes.dark;
2921   return {
2922     algorithm: dark ? antTheme.darkAlgorithm : antTheme.defaultAlgorithm,
2923 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2924     token: {
2925       colorPrimary: p.colorPrimary,
2926       colorInfo: p.colorInfo,
2927       colorSuccess: p.colorSuccess,
2928       colorWarning: p.colorWarning,
2929       colorError: p.colorError,
2930 // [一致] 状态文案与后端枚举对齐
2931       colorBgBase: p.colorBgBase,
2932       colorBgContainer: p.colorBgContainer,
2933       colorBgElevated: p.colorBgElevated,
2934       colorBorder: p.colorBorder,
2935       colorBorderSecondary: p.colorBorderSecondary,
2936       colorText: p.colorText,
2937 // [设计] 状态与视图同步, 保证交互可观测
2938       colorTextSecondary: p.colorTextSecondary,
2939       borderRadius: 10,
2940       borderRadiusLG: 14,
2941       fontFamily:
2942         '"Inter", "SF Pro Display", -apple-system, BlinkMacSystemFont, "Segoe UI", "PingF...',
2943       fontSize: 14,
2944 // [流程] 拉取 → 归一化 → 渲染 → 回写
2945       controlHeight: 38,
2946       boxShadow: p.boxShadow,
2947       wireframe: false,
2948     },
2949     components: {
2950       Layout: {
```

```
2951 // [异常] 失败展示可恢复提示, 避免静默失败
2952     bodyBg: "transparent",
2953     headerBg: "transparent",
2954     siderBg: "transparent",
2955     triggerBg: p.colorBgElevated,
2956 },
2957 Card: {
2958 // [性能] 分页与缓存失效协同, 减少重复请求
2959     colorBgContainer: p.colorBgContainer,
2960     paddingLG: 20,
2961 },
2962 Menu: {
2963     itemBg: "transparent",
2964     subMenuItemBg: "transparent",
2965 // [安全] 鉴权头统一注入, 页面不硬编码密钥
2966     itemSelectedColor: p.itemSelectedColor,
2967     itemHoverBg: p.itemHoverBg,
2968 },
2969 Button: {
2970     primaryShadow: p.primaryShadow,
2971     borderRadius: 10,
2972 // [一致] 状态文案与后端枚举对齐
2973 },
2974 Input: {
2975     activeBorderColor: p.activeBorder,
2976     hoverBorderColor: p.hoverBorder,
2977 },
2978 Tag: {
2979 // [设计] 状态与视图同步, 保证交互可观测
2980     borderRadiusSM: 6,
2981 },
2982 Table: {
2983     headerBg: p.tableHeaderBg,
2984     rowHoverBg: p.rowHoverBg,
2985 },
2986 // [流程] 拉取 → 归一化 → 渲染 → 回写
2987 Modal: {
2988     contentBg: p.colorBgContainer,
2989     headerBg: p.colorBgContainer,
2990 },
2991 },
2992 };
2993 // [异常] 失败展示可恢复提示, 避免静默失败
2994 }
2995 // === 模块: frontend/src/il8n/index.ts ===
2996 // [功能] frontend/src/il8n/index.ts — 工作台前端模块
2997 // [说明] 已脱敏; 保留可审查原创逻辑
2998 /* Lightweight il8n — zh / en with localStorage persistence */
2999 import { create } from "zustand";
3000 // [性能] 分页与缓存失效协同, 减少重复请求
```

```
3001 import { persist } from "zustand/middleware";
3002 export type Locale = "zh" | "en";
3003 const dict = {
3004   zh: {
3005     "nav.dashboard": "总览",
3006     "nav.tasks": "任务",
3007     // [安全] 鉴权头统一注入, 页面不硬编码密钥
3008     "nav.create": "创建任务",
3009     "nav.reports": "报告",
3010     "nav.settings": "设置",
3011     "header.search": "搜索页面 / 任务 / 操作...",
3012     "header.notify": "活动通知",
3013     "header.theme.dark": "切换亮色",
3014     // [一致] 状态文案与后端枚举对齐
3015     "header.theme.light": "切换深色",
3016     "header.workspace": "工作区",
3017     "notify.title": "活动通知",
3018     "notify.active": "进行中",
3019     "notify.empty": "暂无活动",
3020     "notify.allTasks": "查看全部任务",
3021     // [设计] 状态与视图同步, 保证交互可观测
3022     "notify.markAll": "全部已读",
3023     "notify.clear": "清空",
3024     "notify.live": "轮询同步",
3025     "notify.ws": "WebSocket 实时",
3026     "tasks.create": "创建任务",
3027     "tasks.search": "搜索任务名称、描述、Owner...",
3028     // [流程] 拉取 → 归一化 → 渲染 → 回写
3029     "tasks.empty": "没有匹配的任务",
3030     "tasks.view.card": "卡片",
3031     "tasks.view.table": "列表",
3032     "tasks.status.all": "全部状态",
3033     "create.title": "创建评测任务",
3034     "create.subtitle": "配置 Agent 模型与参数, 创建后可导入用例并执行流水线",
3035     // [异常] 失败展示可恢复提示, 避免静默失败
3036     "create.submit": "创建任务",
3037     "create.sample": "填充示例",
3038     "create.back": "返回列表",
3039     "create.cancel": "取消",
3040     "reports.empty": "暂无报告",
3041     "reports.emptyDesc": "完成评测任务后, 终态结果会出现在这里。",
3042     // [性能] 分页与缓存失效协同, 减少重复请求
3043     "reports.view": "查看",
3044     "common.delete": "删除",
3045     "common.detail": "详情",
3046     "common.execute": "执行",
3047     "common.report": "报告",
3048     "dashboard.title": "工作台总览",
3049     // [安全] 鉴权头统一注入, 页面不硬编码密钥
3050     "dashboard.subtitle": "实时掌握评测任务分布与近期活动",
```